CP1 Report 2026 Assignment 1 (20264003, JuneyoungYoon)

All implementations are in script/ and results are in results/

1. Camera RAW Processing and Image Signal Processing

Part 1. Reading RAW Sensor Data

Implementation. rawpy.imread opens the DNG, and a copy of raw_image_visible serves as the 2D Bayer array. It is a monochrome array with one value per pixel, so it is displayed with a gray colormap. The script logs the shape, dtype, CFA indices, black/white levels and camera WB. (script/1/part1.py)

Details.

1. rawpy.imread(path) is opened as a context manager (with) so the file handle is closed. raw_image_visible is the sensor area with the unusable border cropped off. The array is .copy()-ed so it can be used outside the with block.

2. color_desc is a byte string of channel names (b'RGBG'), and the numbers in raw_pattern are indices into it. The second G (index 3) has the same name as the first G (index 1) but a different index.

3. The visualization is plt.imshow(raw_image, cmap='gray'). With one channel there is no color, and each pixel holds only one of R/G/B, which is why a grid texture appears when zoomed in.

4. A log() function writes to both the screen and log.txt.

Output.

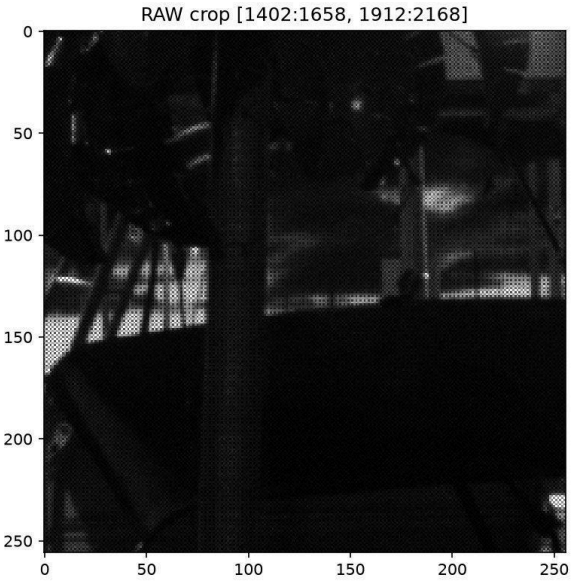
Item	example	example2	example3
shape	3060 × 4080	3060 × 4080	3060 × 4080
dtype	uint16	uint16	uint16
raw_pattern	[[3,2],[0,1]]	[[3,2],[0,1]]	[[3,2],[0,1]]
color_desc	RGBG	RGBG	RGBG
black level (R,G,B,G)	0,0,0,0	0,0,0,0	0,0,0,0
white level	4095	4095	4095
camera WB (R,G,B,–)	1.744, 1.0, 1.943, 0	1.172, 1.0, 2.926, 0	1.825, 1.0, 1.559, 0

The white level is 4095 (2^12 − 1), consistent with a 12-bit sensor, and the metadata reports a black level of 0 for every channel. The array itself is stored as uint16. Since color_desc is RGBG, index 0 = R, 1 = G, 2 = B, 3 = G, and raw_pattern points to the tile [[G,B],[R,G]]. Part 3 verifies this guess directly. The WB gains differ per image because each was shot under different lighting.

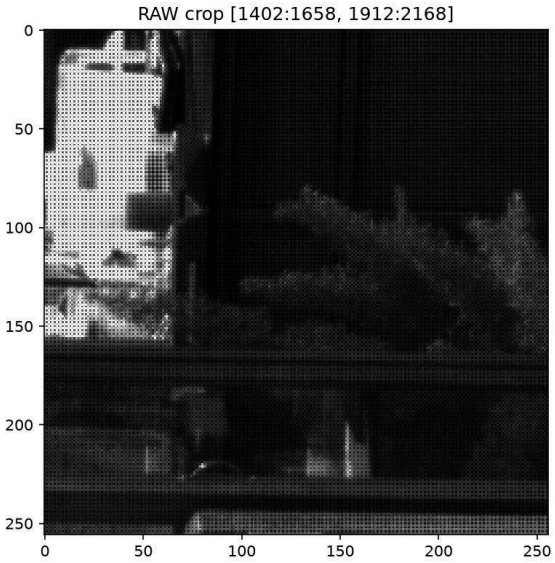
Question. Why does the RAW image contain only one intensity value at each pixel instead of three RGB values?

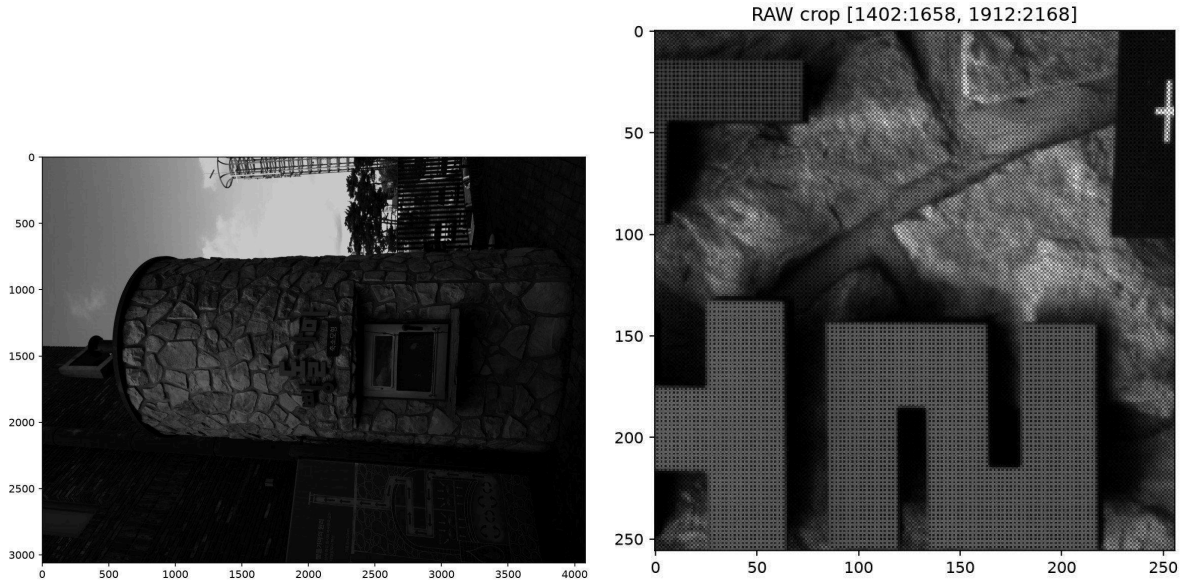
Answer. Each sensor element sits under a single color filter of the color filter array, so it measures only one color component: R, G or B. Using three separate sensors or stacked layers per pixel would be more costly, so cameras sample one color per pixel and later estimate the two missing components by demosaicing. This is why the RAW image is a single-channel array (3060 × 4080) and looks like a gray image with a grid texture.


0
500
1000
1500
2000
2500
3000
3500
4000


RAW crop [1402:1658, 1912:2168]
0
50
100
150
200
250


0
500
1000
1500
2000
2500
3000
3500
4000


RAW crop [1402:1658, 1912:2168]
0
50
100
150
200
250



Part 2. Linearization

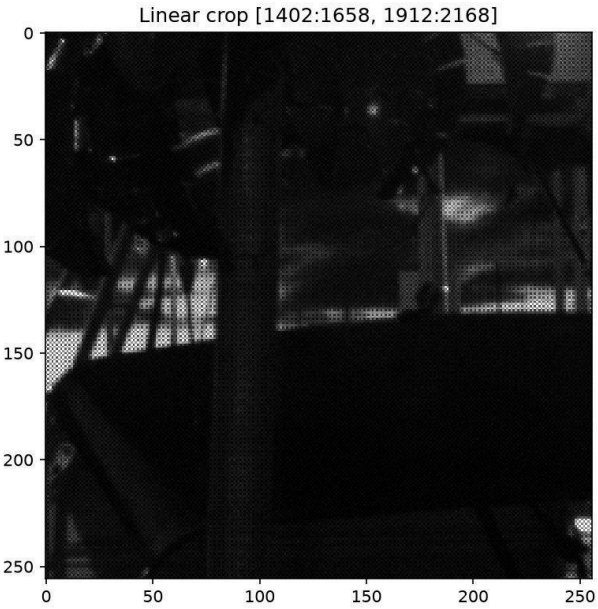
Implementation. normalize computes (raw - black) / (white - black) in float32 and clips to [0, 1]. The black level is expanded from black_level_per_channel into a per-pixel array using raw_colors_visible, so different black levels per Bayer position are also handled.

Details.

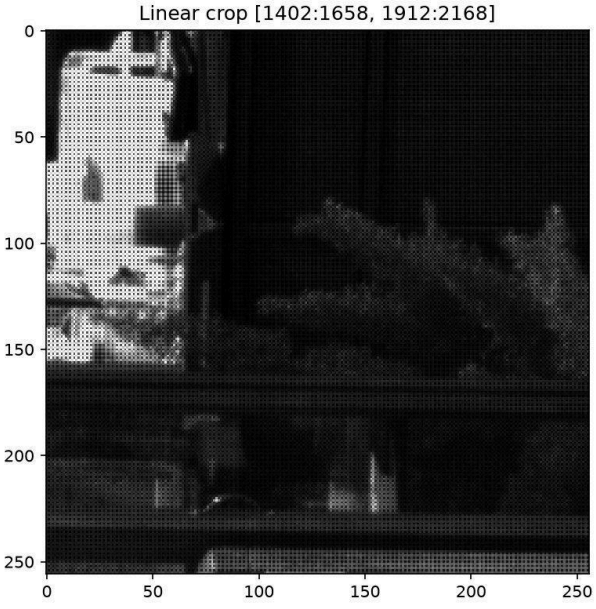
- astype(np.float32) comes first. Subtracting directly in uint16 would overflow instead of going negative for values below the black level.
- black_level_per_channel has 4 entries, and raw_colors_visible is an (H, W) array holding each pixel's channel index (0–3). Indexing with it gives a per-pixel black level (H, W), so one line handles channels with different black levels.
- This sensor's black level is 0, but the code is general enough to work for other cameras.
- The final np.clip guards against noise pushing values below the black level into negatives and floating-point error pushing values slightly above 1. load_linear returns (original, linear image) so other Parts use identical input.

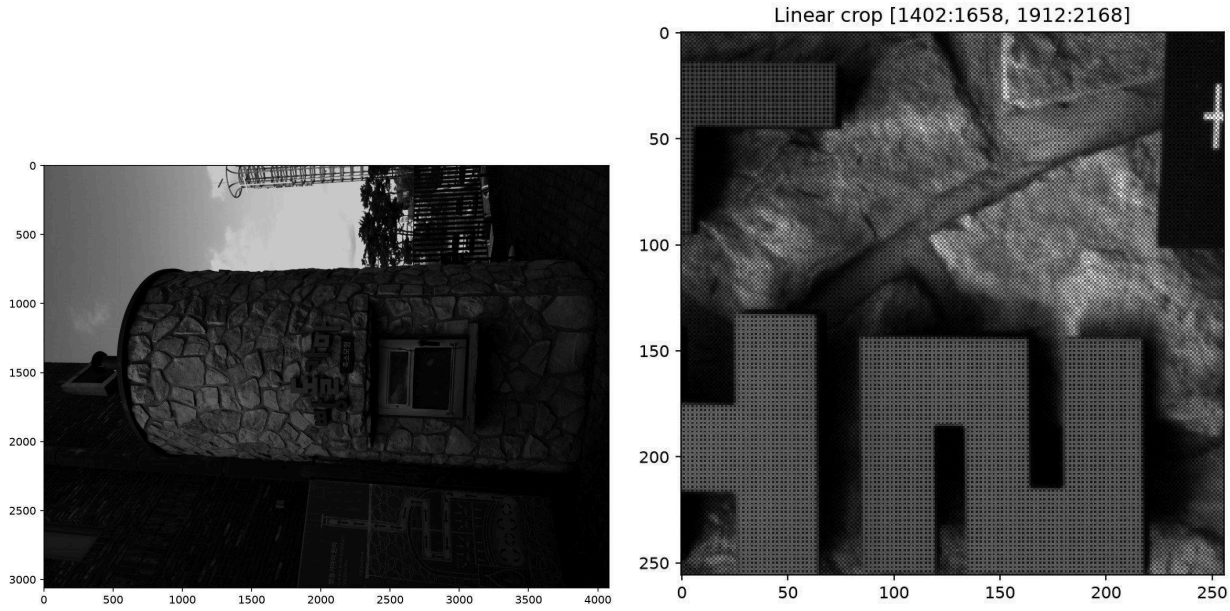
Output. Because the black level is 0, the result equals the raw data divided by 4095, so the Part 1 and Part 2 figures are effectively identical (for example, both files are 431,841 bytes). imshow rescales the value range automatically, so a division by a constant is invisible. The image looks dark and flat because no gamma has been applied yet, which is expected.


0
500
1000
1500
2000
2500
3000
3500
4000


Linear crop [1402:1658, 1912:2168]
0
50
100
150
200
250


0
500
1000
1500
2000
2500
3000
3500
4000


Linear crop [1402:1658, 1912:2168]
0
50
100
150
200
250



Question. Why should black-level correction be performed before subsequent ISP operations? What is the effective bit depth of the RAW image?

Answer. The black level is an offset caused by sensor offset and dark current, so a pixel that received no light does not read 0. White balance gains, the color matrix and the brightness scale are all multiplications, which behave correctly only when values are proportional to light. Applying them before the subtraction would also scale the offset and tint dark regions. Subtracting first restores that proportionality. Here the black level is 0 and the white level is 4095, so the usable range white − black = 4095 holds 4,096 integer levels. The effective bit depth is therefore 12 bits, even though the data are stored as 16-bit integers. A non-zero black level would shrink this range and lower the effective bit depth to log2(white − black + 1).

Part 3. Bayer Pattern

3.1. Determining the Bayer Pattern from the Measurements

Implementation. The linear image is split into the four 2×2 phases (0,0), (0,1), (1,0), (1,1) and their means are compared. G appears twice per tile, so the diagonal pair with similar means is G. To tell which of the remaining two phases is R and which is B, I plotted a log2((0,1)/(1,0)) ratio map over 32×32 block means and judged it on regions whose color is known, such as sky and ground. This gave GBRG, and make_masks builds the M_R, M_G, M_B masks. (script/1/part3_1.py)

Details.

1. **Phase split.** linear[0::2, 0::2], [0::2, 1::2], [1::2, 0::2], [1::2, 1::2] extract the four positions of the 2×2 tile. Each phase holds samples of one color only.

2. **Finding G.** In a Bayer tile G sits on one diagonal. I compare the mean difference of the two diagonals ((0,0)–(1,1) and (0,1)–(1,0)) the one with the smaller difference is the G diagonal. In all three images the (0,0)–(1,1) difference was at most 0.0003.

3. **Separating R and B.** Means alone cannot say which of (0,1) and (1,0) is R, because which one is brighter depends on the scene. So I relied on parts of the scene with known color (sky is blue, ground is roughly neutral). Each phase is reduced to 32×32 block means, and the map of log2(((0,1)+ε) / ((1,0)+ε)) shows which phase responds more strongly in red or blue regions.

4. **Valid blocks.** Dark blocks (G mean below 0.02) and saturated blocks (above 0.9) are excluded (NaN) so noise and clipping do not distort the ratio. The log makes R and B symmetric around zero (a 2× difference becomes +1 or −1).

5. **Mask generation.** make_masks(shape, pattern) converts the pattern string 'GBRG' into a 2×2 tile and tiles it over the whole image using row and column indices modulo 2 (yy, xx). It then compares each pixel's color against 'R', 'G', 'B' to build three boolean masks, which WB and demosaicing use from here on.

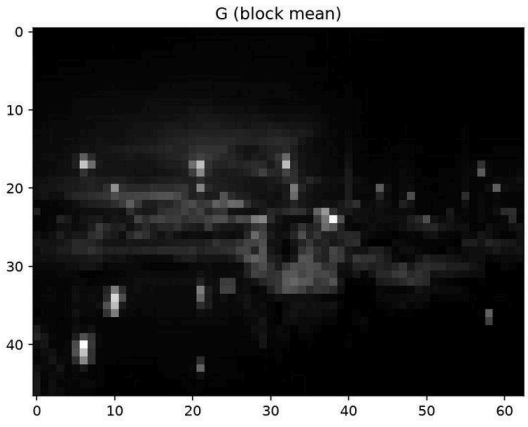
6. **Verification.** assert np.all(M_R.astype(int) + M_G + M_B == 1) confirms every pixel belongs to exactly one mask, and the mask ratios (0.25/0.50/0.25) are logged.

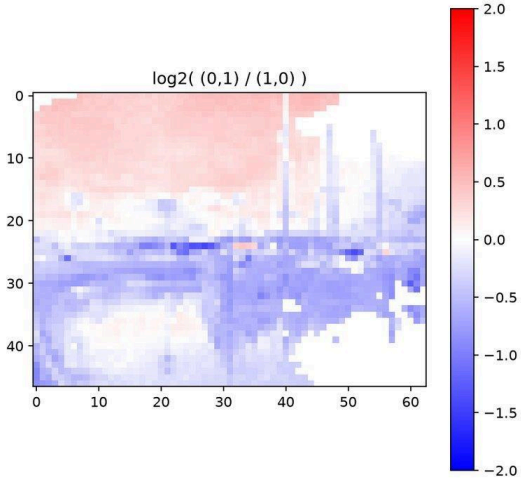
Output.

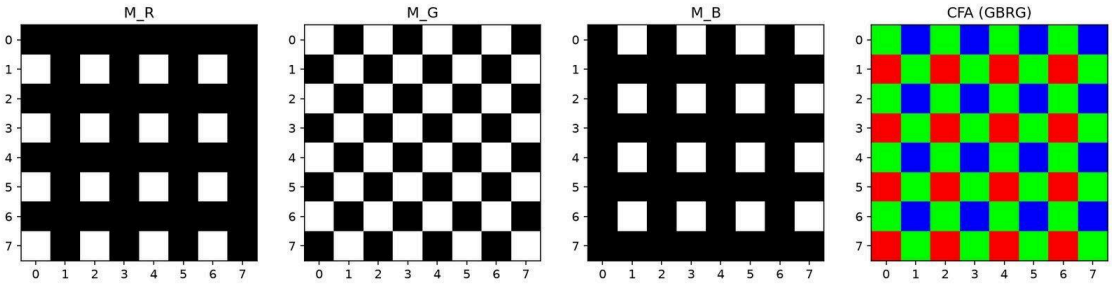
Image	(0,0) mean	(0,1) mean	(1,0) mean	(1,1) mean	diagonal diff (0,0)−(1,1)	diagonal diff (0,1)−(1,0)
example	0.0711	0.0357	0.0418	0.0710	0.0001	0.0061
example2	0.1710	0.0520	0.1590	0.1713	0.0003	0.1070
example3	0.2039	0.1481	0.0980	0.2035	0.0003	0.0501

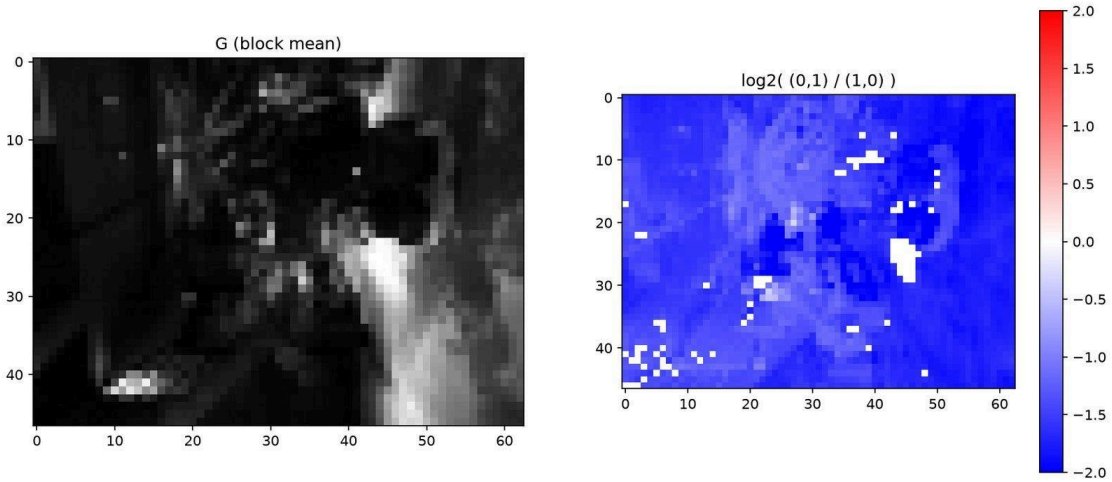
The means of (0,0) and (1,1) differ by at most 0.0003 in all three images, so that diagonal is G. The mask ratios are R/G/B = 0.25/0.50/0.25, and each pixel belongs to exactly one mask.

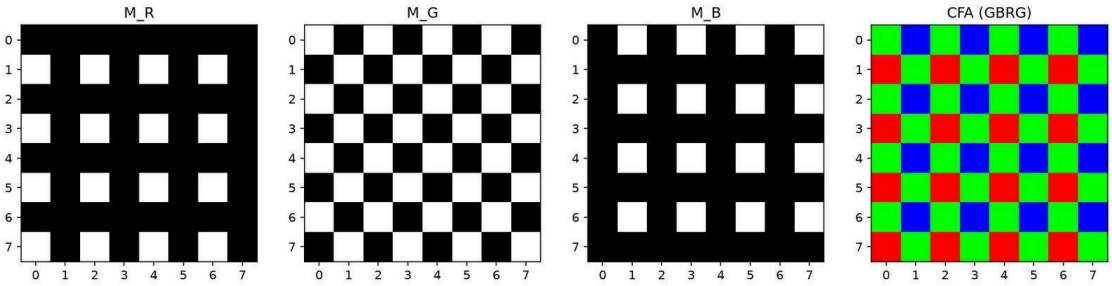
Result for R/B. In the log2((0,1)/(1,0)) map (red = positive, blue = negative; these are colormap colors, not scene colors), the sky at the top of example is positive, so the (0,1) phase responds more strongly there than the (1,0) phase. Since the sky is blue, (0,1) is assigned to B and (1,0) to R. The warm-lit ground, lamp and building regions come out blue (negative), meaning (1,0) is stronger there, which is consistent with (1,0) being R. Together with G on the (0,0)–(1,1) diagonal, the tile is [[G, B], [R, G]], i.e. GBRG.The resulting R, G and B sampling masks are shown below.

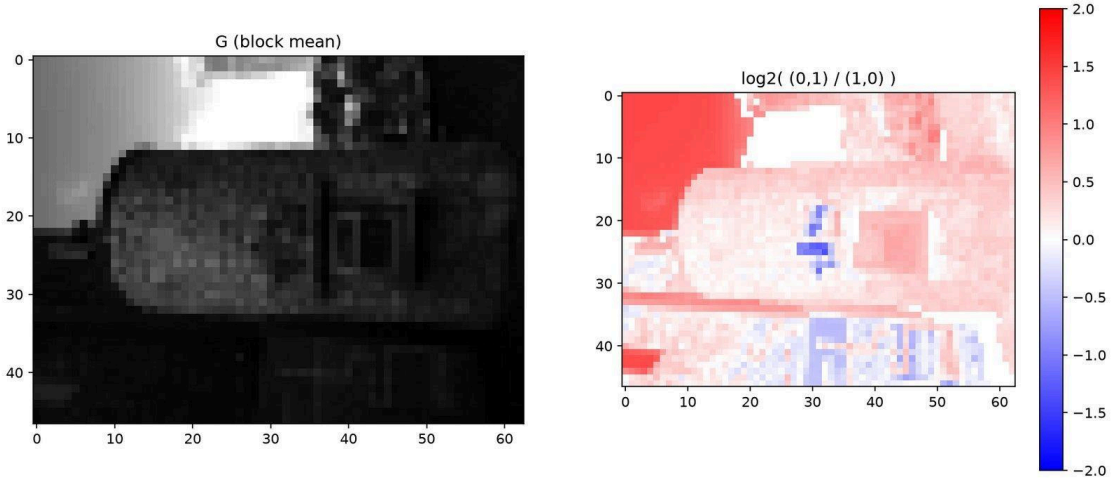


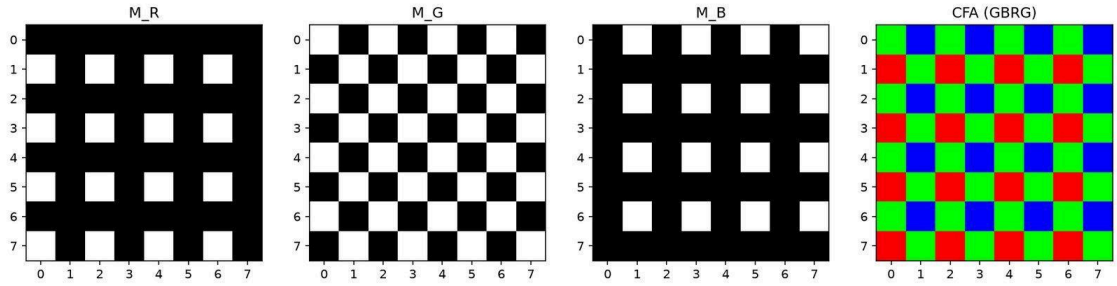



M_R
M_G
M_B
CFA (GBRG)


G (block mean)
log2((0,1) / (1,0))


M_R
M_G
M_B
CFA (GBRG)


G (block mean)
log2((0,1) / (1,0))

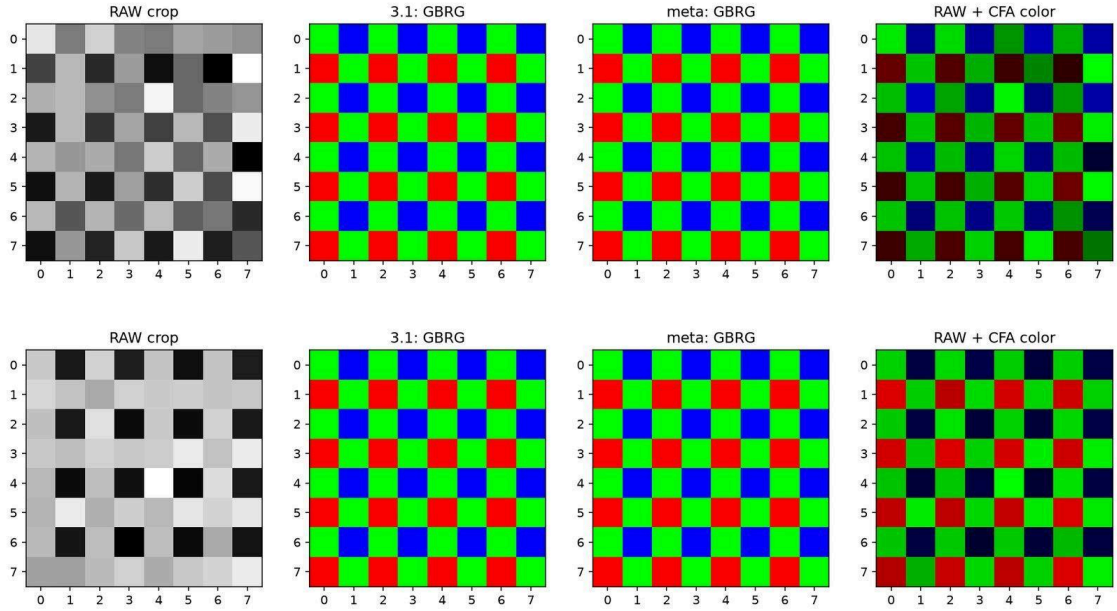


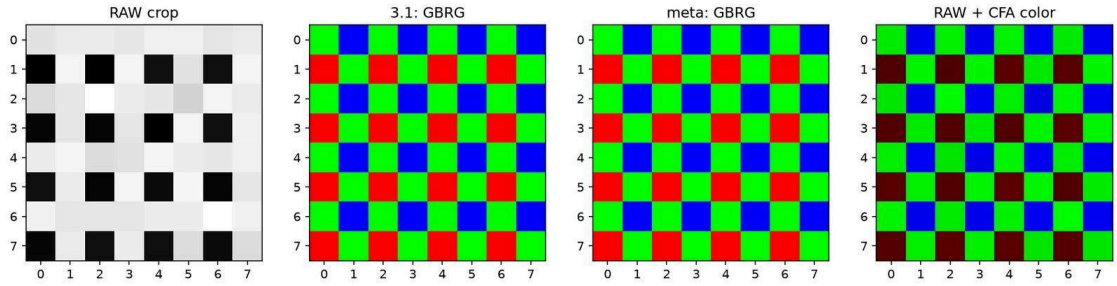
3.2. Comparison with the Bayer Pattern from the Metadata

Implementation. raw_pattern and raw_colors_visible[:2,:2] are mapped to strings through color_desc and compared with the 3.1 result. Besides the strings, the mask arrays themselves are compared with np.array_equal. (script/1/part3_2.py)

Details. to_string flattens a 2×2 index array and joins the matching characters of color_desc.decode() into a string (GBRG). Both metadata sources are checked: raw_pattern refers to the full sensor, and raw_colors_visible[:2,:2] refers to the first 2×2 of the visible area actually used. Because the two can differ (sensors whose crop removes an odd number of rows or columns), the visible-area version is treated as the reference. The comparison is string equality plus np.array_equal on the masks built by make_masks. The visualization places a RAW crop, my masks, the metadata masks and the masks overlaid on the RAW side by side.

Output. For all three images, the 3.1 result = GBRG, metadata (raw_pattern) = GBRG and metadata (visible) = GBRG, and both the string and mask-array comparisons were True. The pattern I detected directly matches the metadata.





Question. Explain how an incorrect Bayer pattern would affect the final RGB image.

Answer. Every sample would be assigned to the wrong color channel. White balance gains would be applied to the wrong pixels, and demosaicing would interpolate each color from the wrong neighbors. Colors then come out swapped or mixed: exchanging R and B turns blue sky orange, and misplacing G gives a strong green or magenta cast. Edges also show color fringing and zipper-like artifacts.

Part 4. White Balance

Implementation. Using the Part 3 masks, a gain map (gR·M_R + gG·M_G + gB·M_B) picks the gain of each pixel's color. It is multiplied into the Bayer image and clipped to [0, 1] (apply_gains). With G fixed at 1, I compared three ways of getting the R and B gains. (script/1/part4.py)

- **Gray-world.** Assumes the scene averages to gray: gain = μ_G / μ_R and μ_G / μ_B.
- **White-world.** Treats the brightest value (100th percentile, i.e. the maximum) as white: gain = max_G / max_R and max_G / max_B.
- **Camera.** Uses the DNG's camera_whitebalance divided by G.

Details.

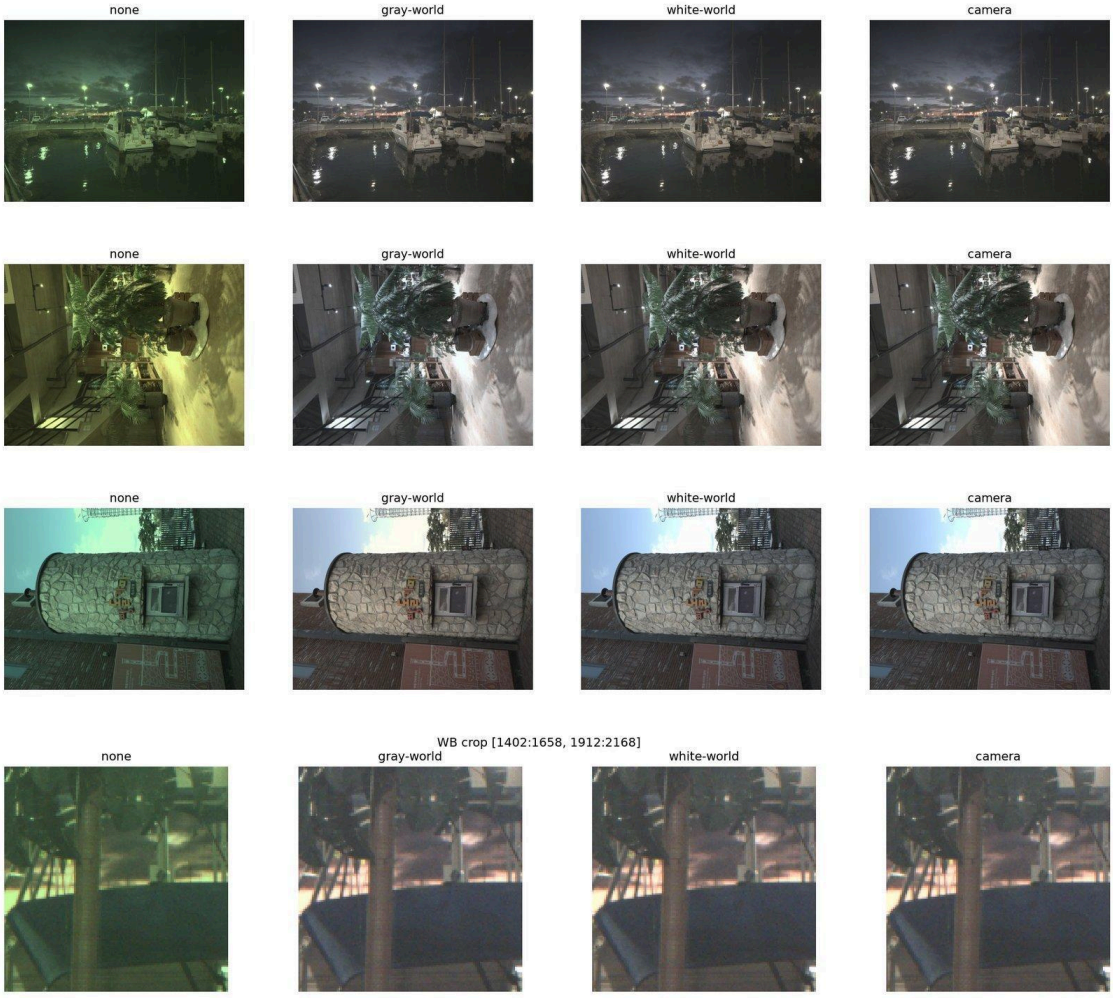
gain_map = gR * MR + gG * MG + gB * MB
out = np.clip(bayer * gain_map, 0, 1)

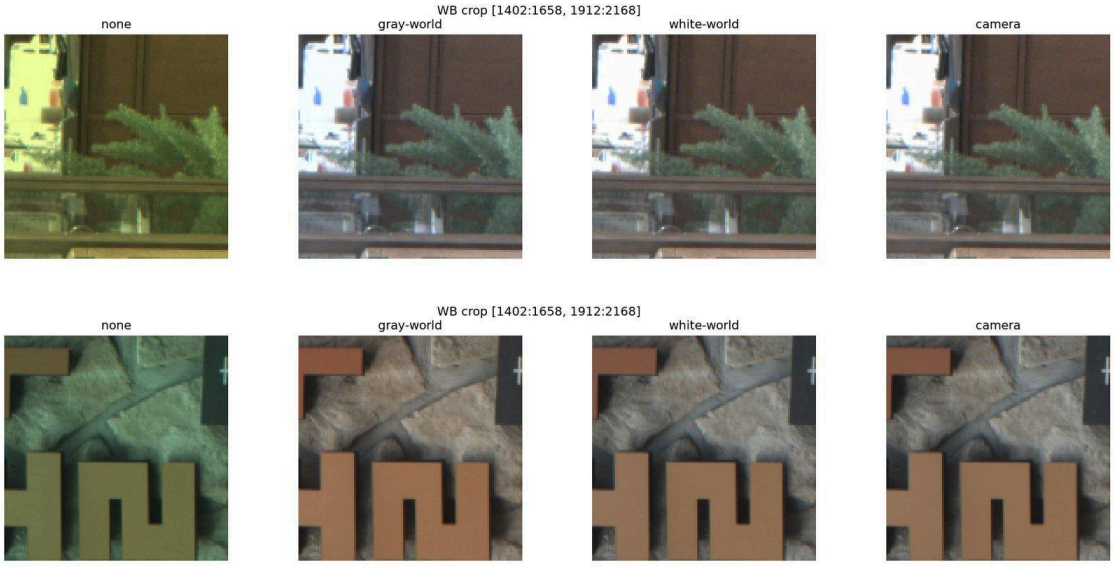
- Multiplying a boolean mask by a scalar gives a float array, so one multiplication replaces a per-channel loop. The result is still Bayer-shaped (H, W). Applying WB before demosaicing is the standard order: it acts directly on the sensor values without affecting interpolated pixels.
- **Gray-world.** mu = [bayer[M].mean() for M in masks] gives the per-channel means and returns (μG/μR, 1, μG/μB), so the channel means become equal before clipping. If the scene is dominated by one color, it mistakes that color for the illuminant.
- **White-world.** np.percentile(bayer[M], WW_PCT) takes the top value per channel. With WW_PCT = 100 this is the maximum. A lower value (e.g. 99) is less sensitive to sensor noise, but at 100 the result was already almost identical to the camera gain. If a channel's maximum is saturated (1.0) the gain can be distorted, but the near-match with the camera gain shows this did not occur in these three images.
- **Camera.** camera_whitebalance uses the same channel order as color_desc (RGBG). The code finds positions with desc.index('R'), desc.index('G'), desc.index('B') and normalizes by G, so it still works if a camera uses another order (e.g. RGGB).
- **Comparison display.** A Bayer image cannot show color, so preview is used: R and B of each tile are kept as they are, the two G values are averaged, giving a half-resolution RGB image with a 1/2.2 gamma. It is only for comparing colors and is not used for the final image.

Output (gain R, G, B).

Method	example	example2	example3
none	1.000, 1, 1.000	1.000, 1, 1.000	1.000, 1, 1.000
gray-world	1.701, 1, 1.992	1.077, 1, 3.291	2.079, 1, 1.375
white-world	1.744, 1, 1.940	1.171, 1, 2.923	1.822, 1, 1.558
camera	1.744, 1, 1.943	1.172, 1, 2.926	1.825, 1, 1.559

The white-world gains differ from the camera gains only in the third decimal in all three images, which suggests that the brightest areas (lights, highlights) are close to neutral. Gray-world is inaccurate when the scene color is skewed: example2's B gain of 3.291 is larger than the camera's 2.926. Visually, the result without WB has a strong yellow or green cast, and all three WB results remove it. Gray-world comes out slightly cooler than camera/white-world. Later Parts use camera WB by default.





Question. Which white-balancing method produces the most natural result? Explain why the results differ.

Answer. Camera white balance gave the most natural result. White-world matched it closely (gains within 0.003 in all three images), while gray-world was the outlier. The methods differ because each assumes something different. Gray-world assumes the scene averages to gray, which fails when one color dominates the scene (example2: B gain 3.291 against 2.926 for the camera). White-world assumes the brightest value is white, which fails if the maximum is saturated or noisy. The camera gains come from the camera's own illuminant estimate at capture time.

Part 5. Demosaicing

Implementation. An averaging-based bilinear interpolation. For each channel, only that color's measured values are kept using its mask, and the missing value is estimated by dividing the 3×3 neighborhood sum by the 3×3 sum of the mask (box3(bayer·M) / box3(M)). Measured pixels are left untouched, and the outside is zero-padded so the denominator stays correct at borders. The output is an (H, W, 3) RGB image, and preservation of measured values is checked with an assert. demosaic_bilinear(bayer, pattern) returns this RGB image. (script/1/part5.py)

Details.

- box3(bayer * M) is the 3×3 neighborhood sum of the image that keeps only that color's measurements, and box3(M) counts how many measurements are in the neighborhood. Their ratio is the mean of the neighboring measurements. The number of samples differs per pixel (at a missing location G has 4 neighbors up/down/left/right, and R/B have 2 or 4 depending on position), so the sum must be divided by the count, not by a constant.

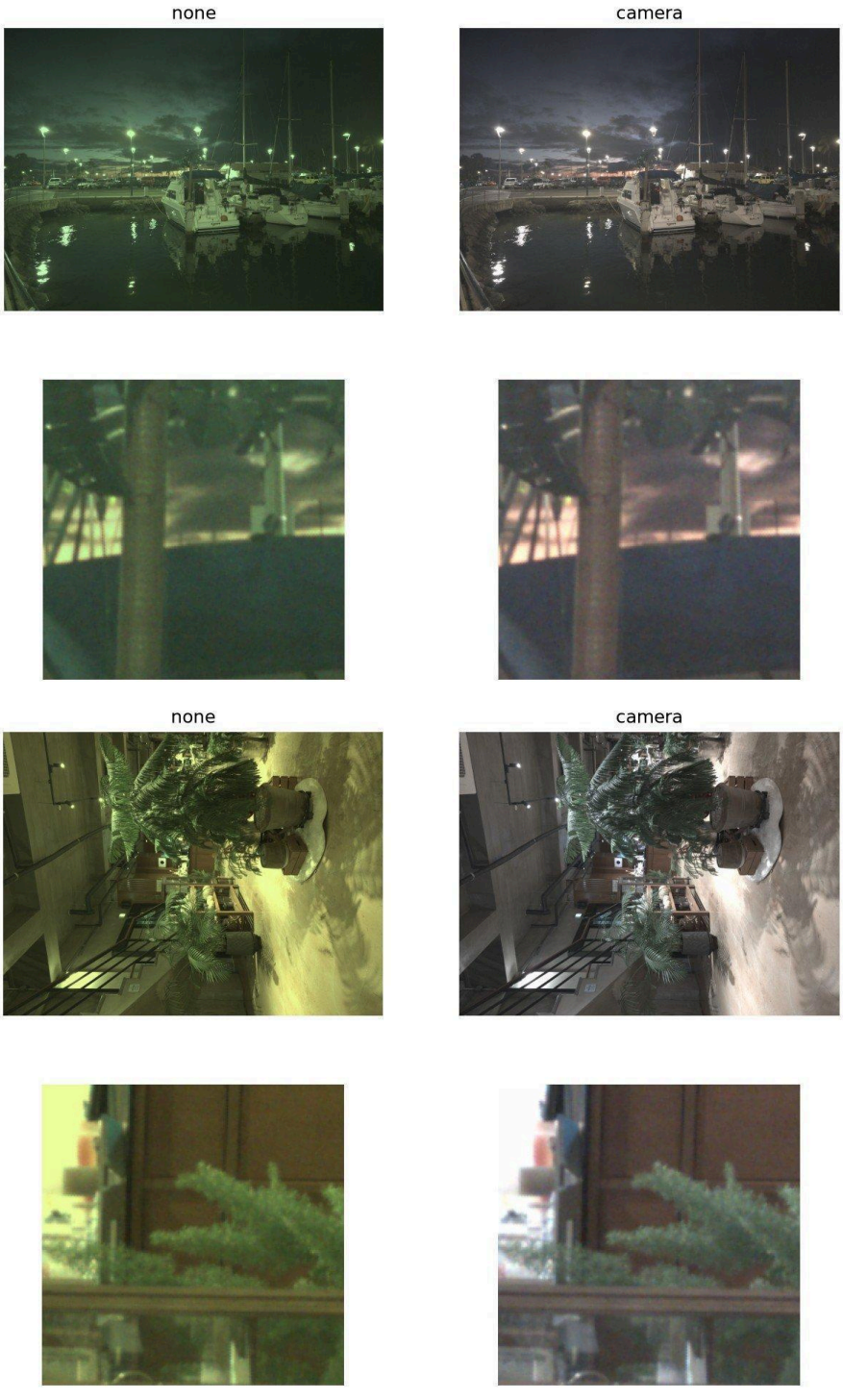
- np.where(M > 0, bayer, est) keeps the measured value rather than an interpolated one at pixels that already have that color. This was checked with assert np.array_equal(rgb[masks[1]][:, 1], bayer[masks[1]]) (G channel).

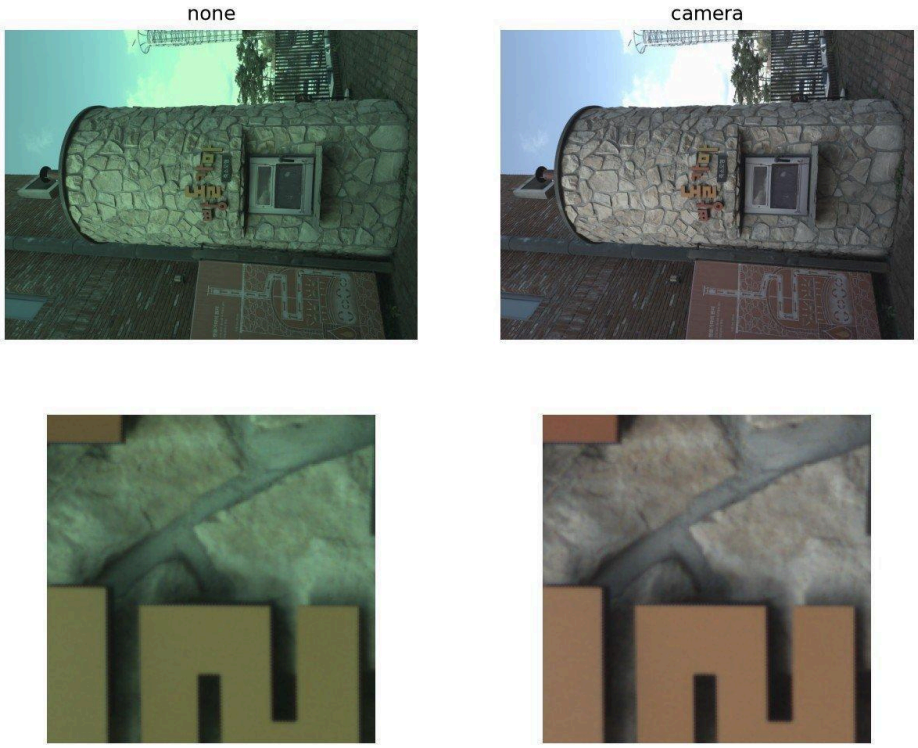
- The padding is zero, but the denominator box3(M) is padded the same way, so at the image border the result is the mean of the neighbors that exist. The denominator never becomes zero (every 3×3 window contains at least one pixel of each color).

- The output is (H, W, 3) in [R, G, B] order. Reusing the 3×3 sum for all channels keeps the coordinate-handling code short. R and B have half the sample density of G (1/4 each), so their interpolated resolution is lower.

- to_display (1/2.2 gamma) is for display only. The final sRGB encoding happens in Part 8.

Output. Results without WB and with camera WB are shown in full and in a central 200×200 crop (display gamma 1/2.2 only). Without WB, example has an overall green cast, and after WB the sky and water return to nearly neutral colors. In the central crop, edges of thin structures look smoothly interpolated and slightly lower in resolution, which is the limitation of bilinear interpolation.





Question. Find a region containing edges or fine textures and describe artifacts produced by bilinear demosaicing.

Answer. Bilinear interpolation is a low-pass operation, so edges and fine textures lose sharpness, as the central crops show for thin structures. R and B are sampled at only one quarter of the pixels each, so near sharp color edges the interpolated values mix samples from both sides of the edge.



Part 6. Camera RGB to Linear sRGB

Implementation. The first 3×3 block of raw.color_matrix (3×4) is used as M_cam→sRGB, and c_sRGB = M · c_cam is computed for every pixel at once with rgb @ M.T. The result is clipped to [0, 1]. The input is camera RGB after camera WB and demosaicing. (script/1/part6.py)

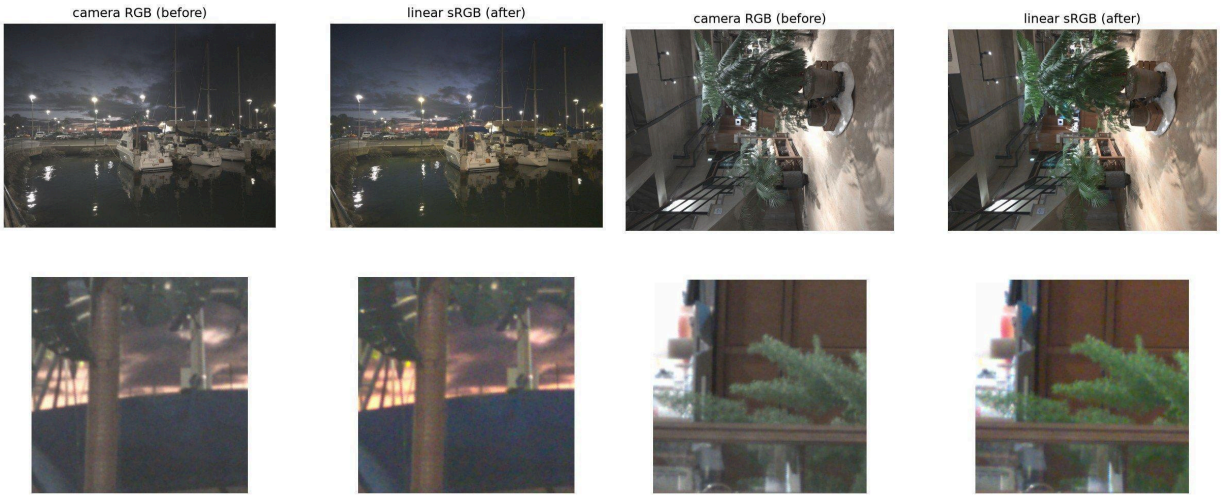
Details.

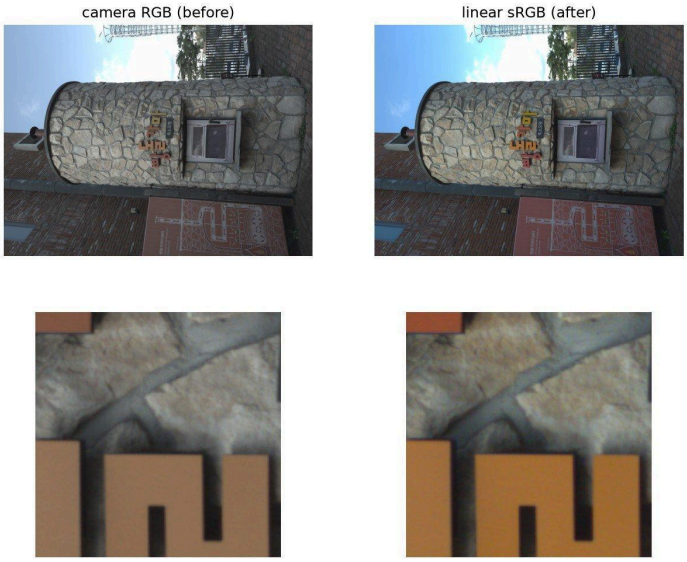
- The formula is c_sRGB = M · c_cam with c_cam as a column vector. The image is (H, W, 3), i.e. row vectors, so the matrix is transposed and written rgb @ M.T. This applies to all pixels without a loop, omitting the transpose would effectively transpose the matrix and change the colors.
- raw.color_matrix is reported as 3×4, but only the first 3×3 is used. It is read from the DNG metadata. The matrix is the same for all images because they come from the same camera.
- **Step order.** The matrix must be applied to channel values after white balance, so the order is camera WB → demosaicing → color conversion. The row sums are logged to check that neutral is preserved (all 1).
- The matrix has negative entries, so saturated colors can leave [0, 1], hence np.clip. The downside is that clipping can lose information in highly saturated colors.

Output. The matrix is identical for all three images (same camera).

$$M_{cam\to sRGB} = [[1.5959,\ -0.4339,\ -0.1620],\ [-0.1946,\ 1.4224,\ -0.2278],\ [0.0121,\ -0.7481,\ 1.7359]]$$

The row sums are [1.0000, 1.0000, 1.0000], meaning white (1,1,1) stays (1,1,1) after conversion. So converting colors after WB keeps neutral colors neutral. The large diagonal entries and negative off-diagonal entries subtract cross-talk between channels, which increases saturation.





Question. Why does the demosaiced camera RGB image have incorrect colors even after white balancing?

Answer. White balancing only multiplies each channel by its own gain, a diagonal transform that neutralizes the illuminant. It does not fix the fact that the sensor's spectral sensitivities differ from the sRGB primaries, so each camera channel responds to a mixture of the sRGB colors. A full 3×3 matrix with off-diagonal terms is needed to map camera RGB to sRGB. The negative off-diagonal entries of the matrix above remove this cross-talk, which is why colors look washed out before the correction and more saturated after it.

Part 7. Brightness Adjustment

Implementation. Luminance is computed from linear sRGB with the Rec.709 weights (0.2126, 0.7152, 0.0722). A scalar α = m / mean(I_gray) makes the mean equal a target m. it multiplies all channels and values above 1 are clipped. I compared m = 0.15, 0.20, 0.25, 0.30. "Clipped" is the fraction of pixels where at least one channel reaches 1. (script/1/part7.py)

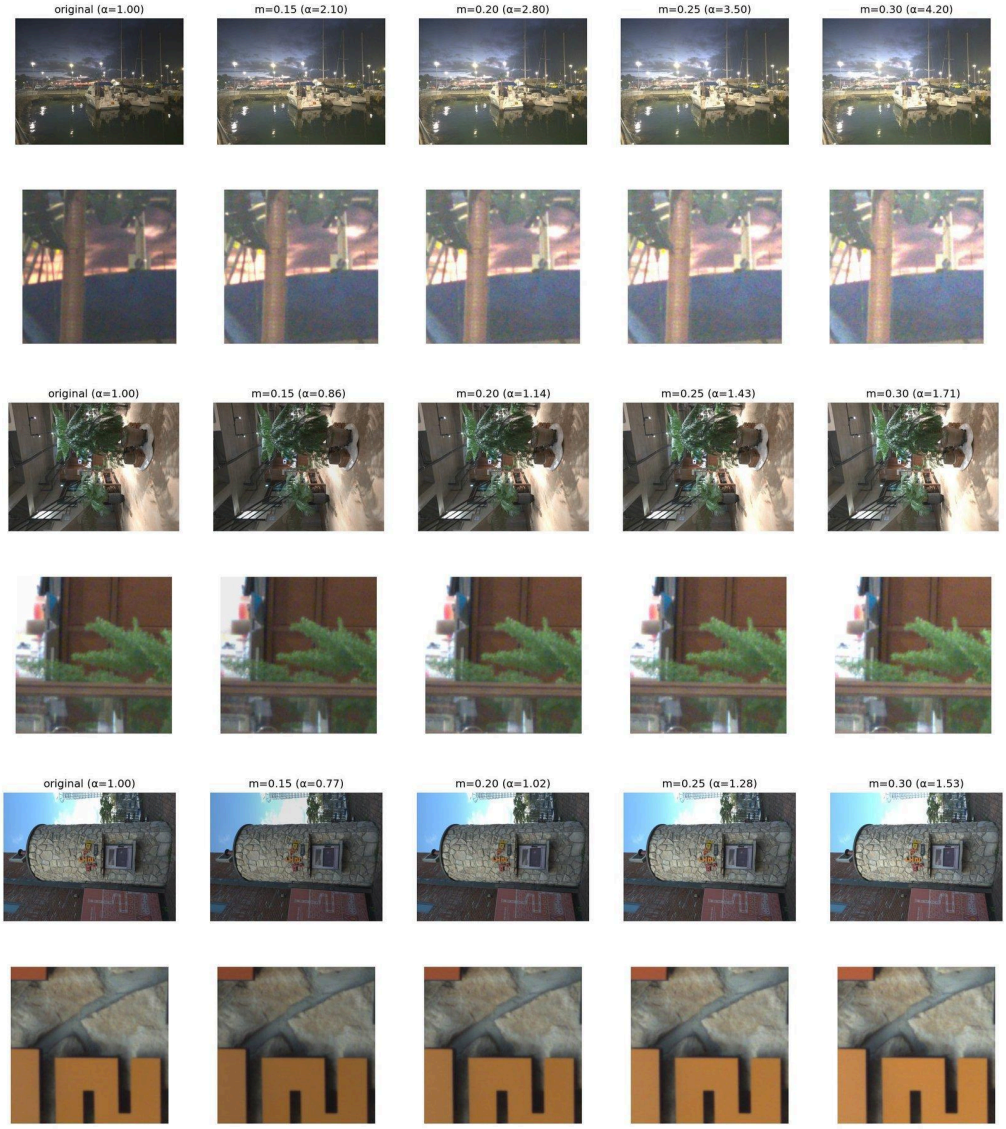
Details.

- **Linear space.** Brightness must be adjusted in linear space: multiplying by a scalar is equivalent to increasing the amount of light. Multiplying after gamma gives a different contrast than intended.
- **Luminance.** Instead of a plain channel mean, the Rec.709 weights (R 0.2126, G 0.7152, B 0.0722) match human brightness perception. These are the weights for linear sRGB.
- **Scaling.** α = m / mean(I_gray) makes the luminance mean exactly m before clipping, and slightly lower after clipping. That is why the logged mean gray is below the target (e.g. 0.2664 for m = 0.30 on example3).
- **Clipping ratio.** (img >= 1).any(axis=-1).mean() gives the fraction of pixels with at least one channel at 1, which quantifies the highlight loss when brightening. It also shows that some pixels are already clipped in the original (α = 1).
- The pipeline default is TARGET = 0.20. After encoding the mean is around 0.40, a normal brightness on screen.

Output (α / clipped).

Setting	example	example2	example3
original (mean gray)	0.0715 / α=1 / 0.67%	0.1753 / α=1 / 3.12%	0.1959 / α=1 / 10.63%
m = 0.15	α=2.098 / 1.73%	α=0.856 / 0.00%	α=0.766 / 0.00%
m = 0.20	α=2.798 / 3.11%	α=1.141 / 4.82%	α=1.021 / 11.21%
m = 0.25	α=3.497 / 4.71%	α=1.426 / 7.92%	α=1.276 / 16.68%
m = 0.30	α=4.196 / 6.58%	α=1.712 / 10.51%	α=1.532 / 16.88%

The dark example gets brighter (α > 1), while the already bright example2 and example3 get darker at m = 0.15 (α < 1). As m grows, the clipped fraction grows and highlights are lost. example3 in particular is already 10.6% clipped, so it has little headroom. The rest of the pipeline uses m = 0.20 as a compromise between brightness and highlight preservation.



Result. The target m = 0.20 gives the most visually pleasing result. The dark example reaches a normal brightness, and the clipped fraction stays at 3–11%, while m = 0.25 and 0.30 clip noticeably more highlights.

Part 8. sRGB Gamma Encoding

Implementation. The piecewise sRGB transfer function encodes linear values into non-linear ones: 12.92·c for c ≤ 0.0031308, otherwise 1.055·c^(1/2.4) − 0.055. This is the final encoding, not the 1/2.2 display gamma, so no further gamma is applied for display. (script/1/part8.py)

Details.

- np.where computes both branches and then selects. The power term c ** (1/2.4) gives NaN for negative input, so the input is clipped to [0, 1] first.
- The straight line 12.92·c is used near 0 because the slope of a power function goes to infinity at 0. The constants are chosen so the two formulas meet at 0.0031308. Away from zero the curve is close to a 2.2 gamma.
- Gamma encoding is placed last because all earlier operations (WB, interpolation, matrix, brightness scaling) are physically meaningful only on linear responses. The comparison figures show before/after encoding side by side, and transfer_curve.png plots the transfer function.

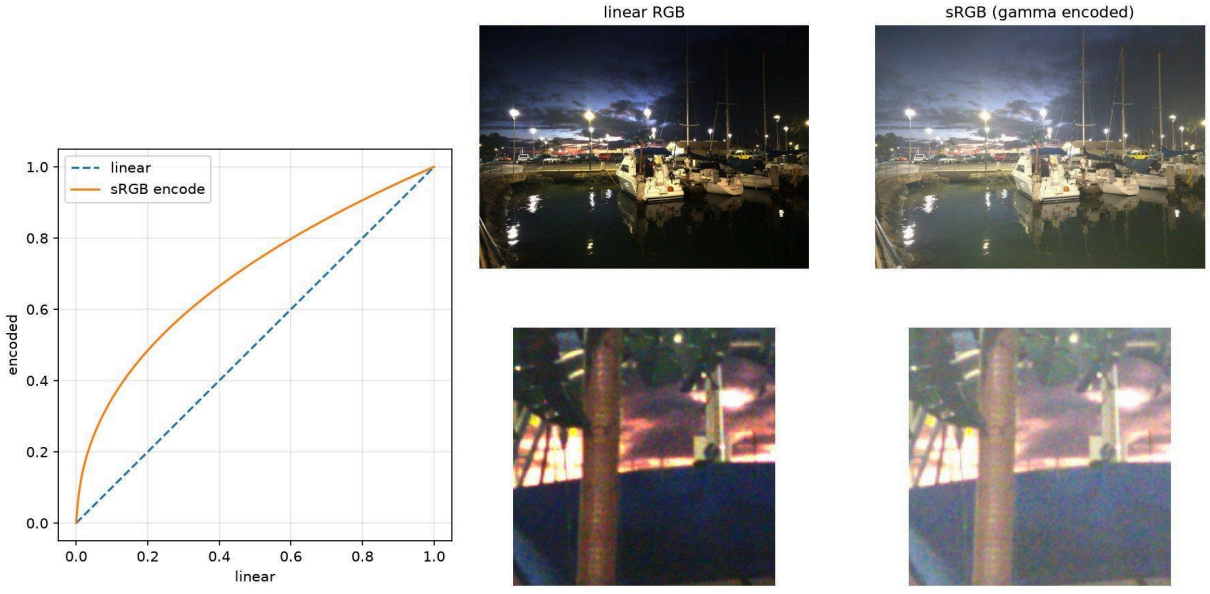
$$C_{sRGB} = 12.92 \times C_{lin}, \quad \text{if } C_{lin} \leq 0.0031308$$

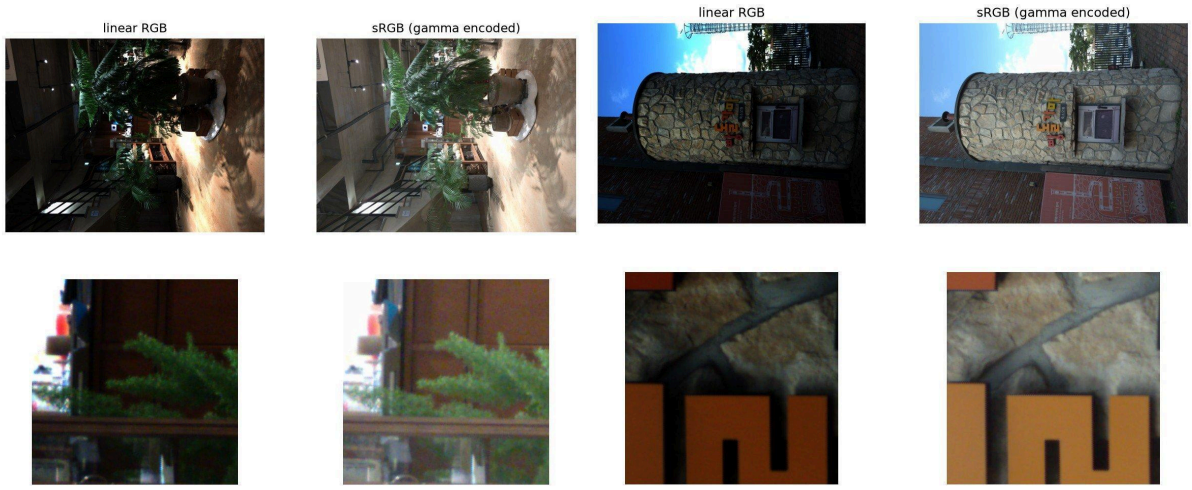
$$C_{sRGB} = 1.055 \times C_{lin}^{(1/2.4)} - 0.055, \quad \text{if } C_{lin} > 0.0031308$$

Output.

Image	mean (linear)	mean (encoded)
example	0.1832	0.4137
example2	0.1902	0.3976
example3	0.2047	0.4048

Encoding raises the mean from 0.18–0.20 to 0.40–0.41. The dark range is stretched upward, giving a tone that looks natural to the eye. A linear image shown directly on screen looks dark with excessive contrast





Question. Why does a linear RGB image appear dark when it is directly displayed on a standard monitor?

Answer. A standard monitor does not respond linearly to its input: its output intensity is roughly the input raised to a power of about 2.2. Sending linear values directly therefore darkens the mid-tones (a linear value of 0.18 is displayed at about 0.02 of full brightness). The sRGB encoding applies the inverse curve beforehand so the displayed result matches the intended brightness. Here it raises the mean from 0.18–0.20 to 0.40–0.41.

Part 9. Complete Camera ISP Pipeline

Implementation. The functions from Parts 2–8 are combined into one camera_isp(raw_image, metadata). The pixel data is an argument, and the image-dependent values (black/white level, WB, color matrix, Bayer pattern, WB method, target brightness) are collected into a dict by get_metadata. The steps run in the order below, and return_stages=True returns the intermediate result of each step. The Bayer pattern is the one I detected in Part 3.1 (GBRG). (script/1/part9.py)

1. linearize: black level removal + normalization (Part 2)
2. white_balance: camera / gray / white (Part 4)
3. demosaic_bilinear: Bayer → RGB (Part 5)
4. color_correction: camera RGB → linear sRGB (Part 6)
5. adjust: brightness adjustment, m = 0.20 (Part 7)
6. srgb_encode: gamma encoding (Part 8)

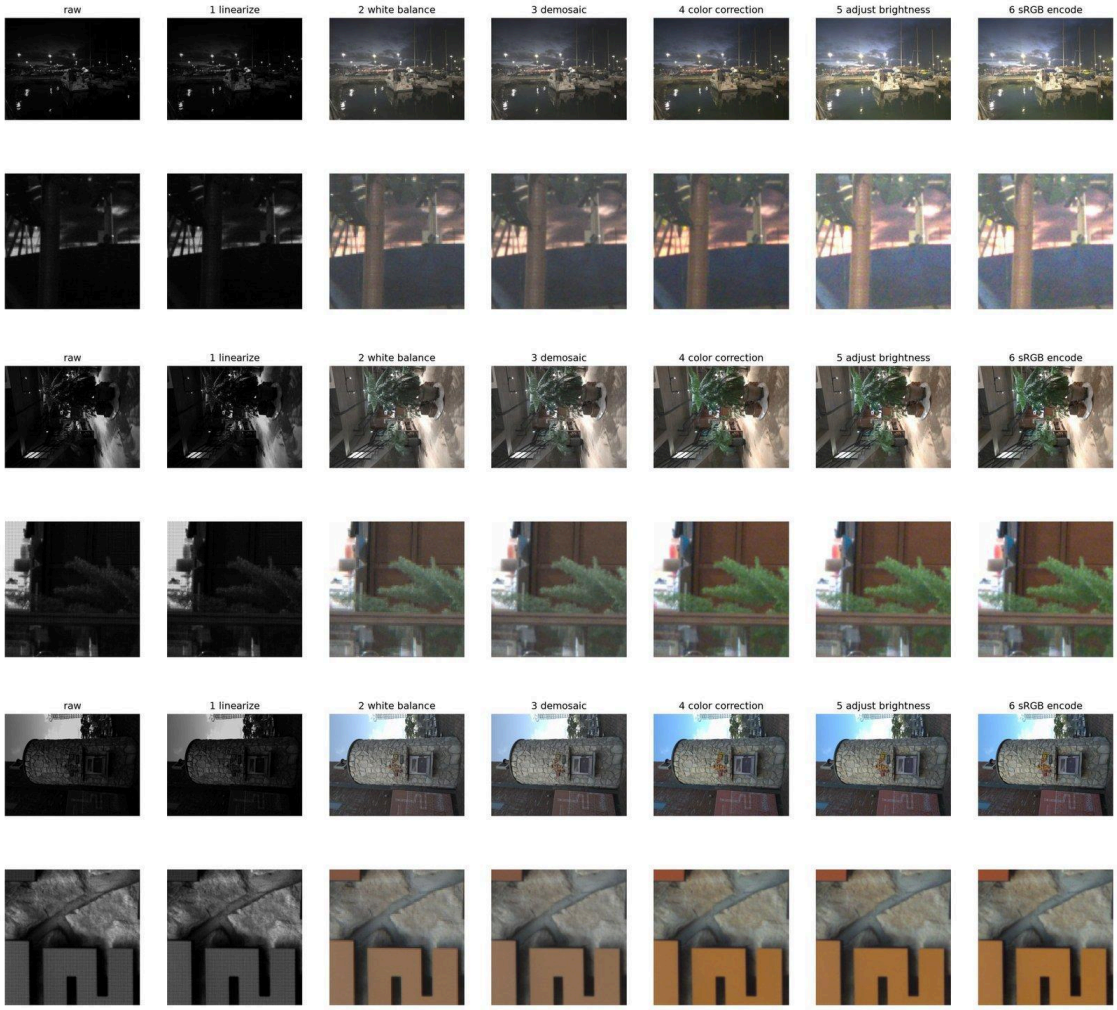
Details.

- **Metadata dict.** get_metadata(raw, wb_method) builds a dict with black (per pixel), white, wb, color_desc, cam2srgb, pattern, wb_method and target. The pipeline functions no longer take a rawpy object, they work from this dict and the raw array alone. So they can run after the file handle is closed, and other settings can be tried by changing only the metadata.
- **WB selection.** white_balance computes gains from the image if metadata['wb_method'] is 'gray' or 'white', and otherwise uses the camera gains. Changing one line therefore produces the final image for each of the three WB methods.
- **Stage shapes.** Stages 1 and 2 are (H, W) Bayer stage 3 onward is (H, W, 3) RGB. Each is visualized differently: raw and stage 1 in gray, stage 2 with preview (2×2 binning), stages 3–5 with the 1/2.2

display gamma, and stage 6 as is. Showing every intermediate the same way would distort the differences between steps.

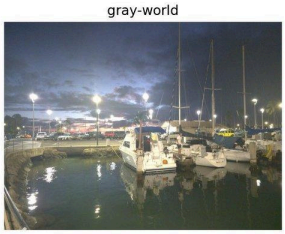
- **Size check.** assert final.shape == (H, W, 3) verifies the final output shape. The final image is saved with plt.imsave and used for the Part 10 comparison.

Outputs. Three sets of figures were produced for each image: (1) every stage from raw to the final result, in full and as a central crop (the WB stage is a 2×2-binned preview because it is still Bayer, and the still-linear stages 3–5 are shown with display gamma) (2) the final result with only the WB method changed (gray-world, white-world, camera) and (3) the final image with camera WB



Comparison Result: WB all remove the green cast seen in the unbalanced result, and the plants, wall and floor look natural in all three. Differences are gray-world is slightly cooler, while white-world and camera are almost identical.

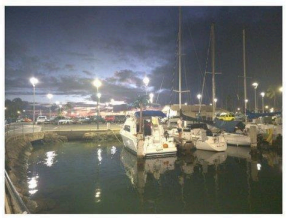
gray-world

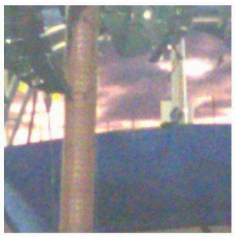


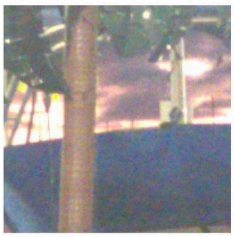
white-world

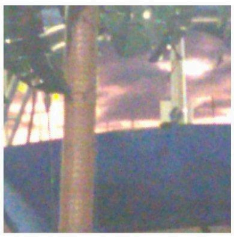


camera

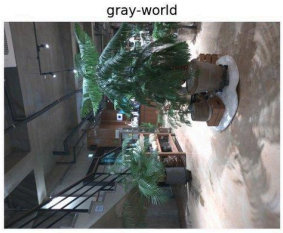




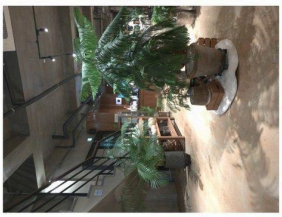




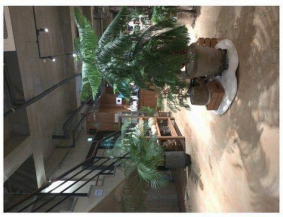
gray-world

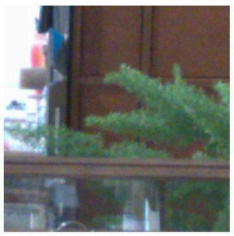


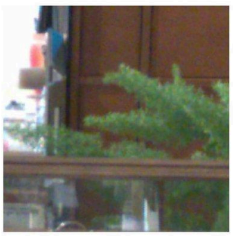
white-world



camera

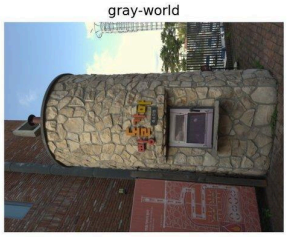




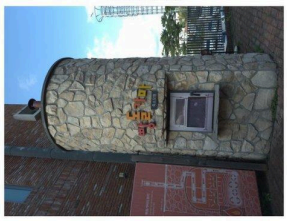




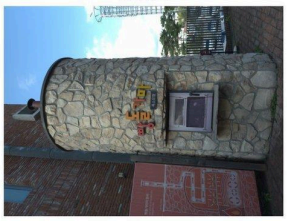
gray-world



white-world

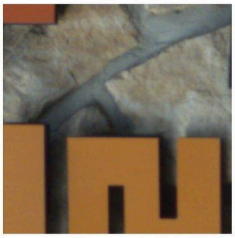


camera

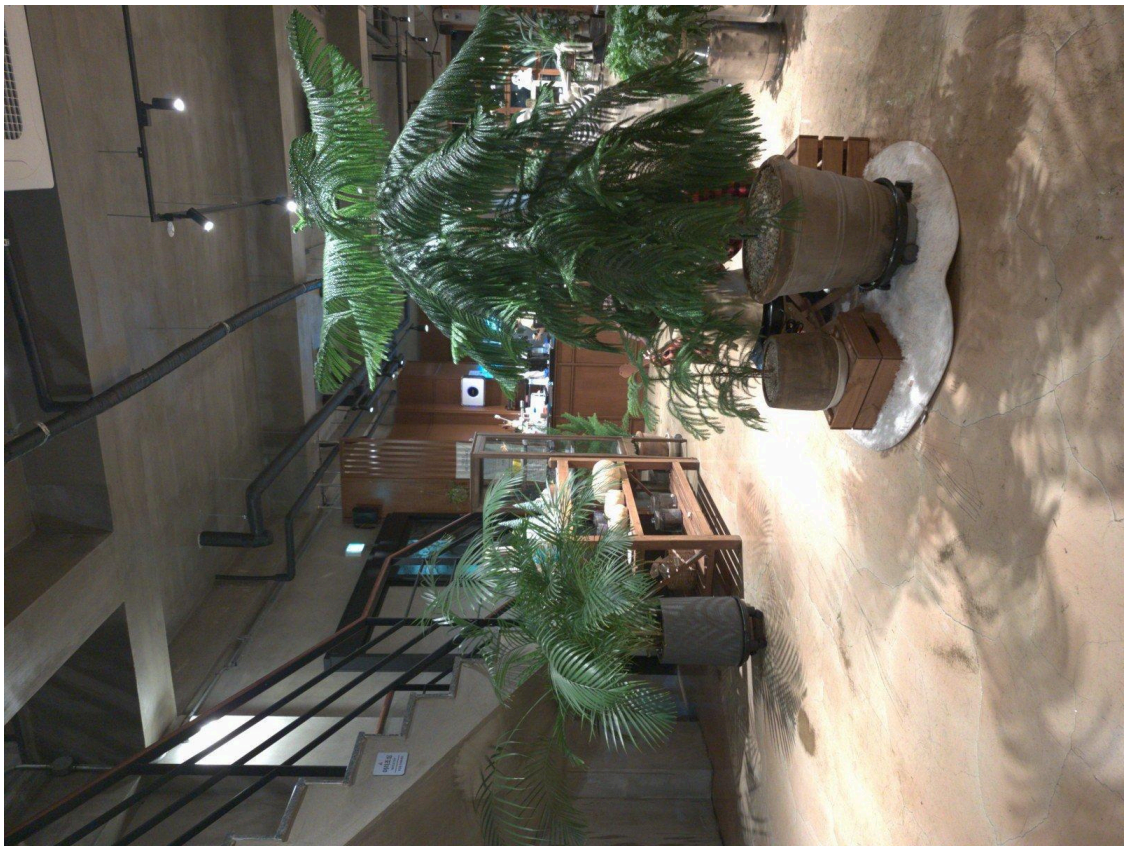














Part 10. Comparison with RAW Processor and Camera JPEG

Implementation. My result is compared with raw.postprocess(use_camera_wb=True, output_bps=16) and with the JPEG produced by the camera. postprocess is used only in this Part. The quantitative metrics are mean brightness (mean luminance), contrast (standard deviation of luminance) and sharpness (mean gradient magnitude of luminance). Because the three results have different resolutions, the central crop is specified as a fraction of image size (8%) so that roughly the same central region is compared. (script/1/part10.py)

Details.

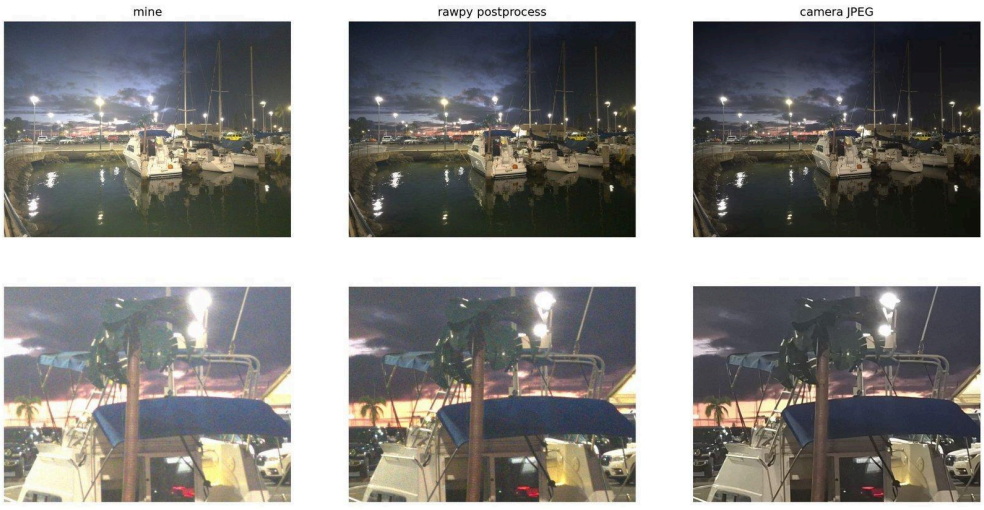
- **What is compared.** (1) the camera_isp result, (2) the 16-bit result of raw.postprocess(use_camera_wb=True, output_bps=16) divided by 65535 into [0, 1], and (3) the camera JPEG (plt.imread) divided by 255 into [0, 1]. All three are compared on the same range.
- **Metrics stats(img).** From grayscale g = rgb2gray(img) it computes the mean (brightness), standard deviation (contrast) and sharpness. Sharpness is the mean of the gradient magnitude $\sqrt{(gx^2 + gy^2)}$, where gx, gy are horizontal and vertical pixel differences from np.diff, the last row and column are dropped so the two differences have the same size. Since images of different resolution are compared, these metrics show relative trends rather than absolute values.
- **Crop.** center_crop cuts the center as a fraction of image size (CROP_FRAC = 0.08). The DNG (3060×4080), rotated rawpy (4080×3060) and JPEG (3000×4000) differ in size, so using the same pixel coordinates would cover different scene areas.

Output.

Image	Method	mean brightness	contrast (std)	sharpness
example	mine	0.417	0.180	0.0240
example	rawpy postprocess	0.265	0.162	0.0264
example	camera JPEG	0.217	0.146	0.0148
example2	mine	0.408	0.223	0.0219
example2	rawpy postprocess	0.332	0.229	0.0273
example2	camera JPEG	0.356	0.239	0.0266
example3	mine	0.402	0.232	0.0140
example3	rawpy postprocess	0.347	0.247	0.0164
example3	camera JPEG	0.370	0.253	0.0289

Analysis.

- Mean brightness is highest for mine in all three images (0.40–0.42). This is because the linear luminance is scaled to m = 0.20, and sRGB encoding then brings it to about 0.40. rawpy and the camera JPEG render darker (especially example).
- Contrast for mine is higher on example and slightly lower on example2 and example3. The largest gap among the three methods is about 0.03 (example: mine 0.180 vs JPEG 0.146).
- Sharpness on example is rawpy (0.0264) > mine (0.0240) > JPEG (0.0148), while on example3 the JPEG (0.0289) is highest. The camera JPEG includes its own noise reduction and sharpening, so the trend is not consistent.
- Overall, I think all three images are quite good. However, the others are a bit too dark, causing some details to get mushed, so I find my image preferable because it keeps those details visible.





Question. Which of the three results do you prefer, and what processing differences do you think are responsible for their different appearances?

Answer. I prefer my image because it's brighter, which lets you see details like the boat in dark scenes or the ceiling indoors much better. The differences come mainly from the tone curve(sRGB) and from demosaicing (bilinear) which is different from rawpy . My result is the brightest because the linear luminance is scaled to 0.20 before sRGB encoding, whereas rawpy and the camera JPEG are darker. The camera JPEG also includes its own noise reduction and sharpening, which explains why its sharpness does not follow the same trend as the others.

2. Image Filtering

The inputs are the three camera JPEGs. They are converted to grayscale (0.299, 0.587, 0.114) and kept at full resolution (4000 × 3000, no downsampling) (load_gray). For zoomed-in views, each figure also has a central 128 × 128 crop (crop_slice) the convolution is computed on the whole image and then cropped, so border effects are excluded.

Part 1. Basic Filtering with 2D Convolution

Implementation. conv2d pads the image by the kernel radius with symmetric padding and flips the kernel. For each kernel element (i, j), it multiplies the padded image shifted by (i, j) by the weight and accumulates. Instead of a pixel loop, this is k^2 vectorized additions. The output has the same size as the input (same). The box filter is a k×k kernel with every element $1/k^2$. I compared the result and running time against scipy's convolve2d(mode='same', boundary='symm') (minimum of 3 runs). (script/2/part1.py)

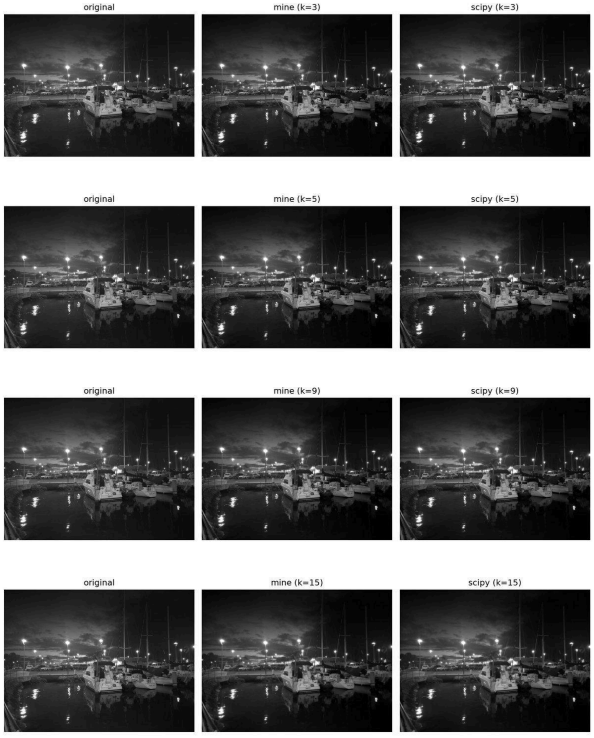
Details.

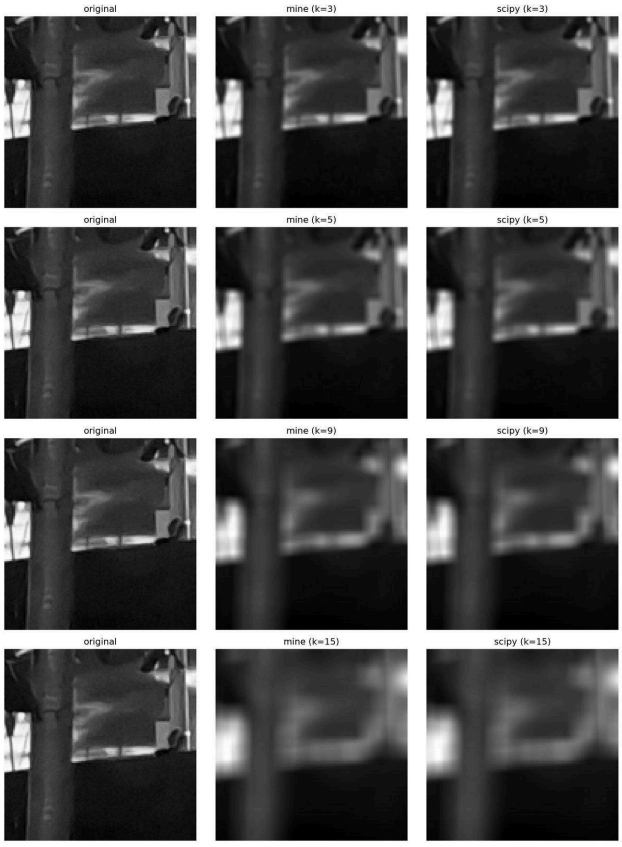
- **Padding.** The image is padded by the kernel radius (k // 2) on every side so the kernel center can sit on edge pixels, which makes the output (h, w) like the input. Odd kernel sizes are assumed. symmetric reflects the border like a mirror and prevents the border from darkening
- **Flipping.** Convolution applies the kernel flipped up-down and left-right. without the flip it would be correlation. For symmetric kernels such as box and Gaussian the result is the same, but for asymmetric kernels such as Sobel the sign can change, so I flipped it as the definition requires.
- **Accumulation.** Output pixel (y, x) is Σ kernel[i, j] · padded[y+i, x+j]. I compute it by adding, one kernel element at a time, a whole-image term (padded[i:i+h, j:j+w] is the image shifted by that offset). Compared with a four-level pixel loop, this needs only k^2 Python iterations and uses numpy vector operations. At full resolution it is slower than scipy for $k \geq 5$ (see the timing table), because the Python loop adds one full-image pass per kernel element.
- **Full resolution and crop.** load_gray does not downsample: the images are processed at the full 4000 × 3000 resolution. For zoomed-in views, crop_slice returns a central 128 × 128 window (CROP = 128), shown with nearest-neighbor interpolation so individual pixels are visible. All Parts use the same two functions, so results are comparable.
- **Timing.** timeit runs the same function 3 times and returns the fastest time from perf_counter (the minimum is less affected by other processes). The error is np.abs(mine - ref).max().

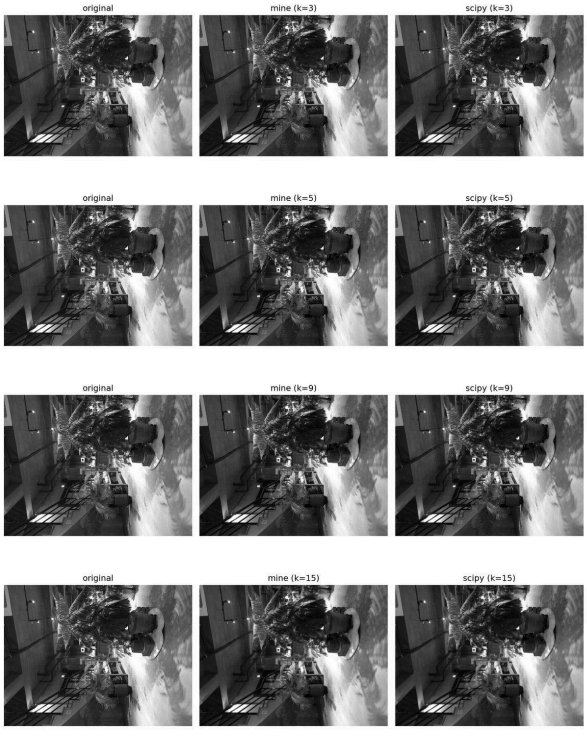
Output (example, running time and max error).

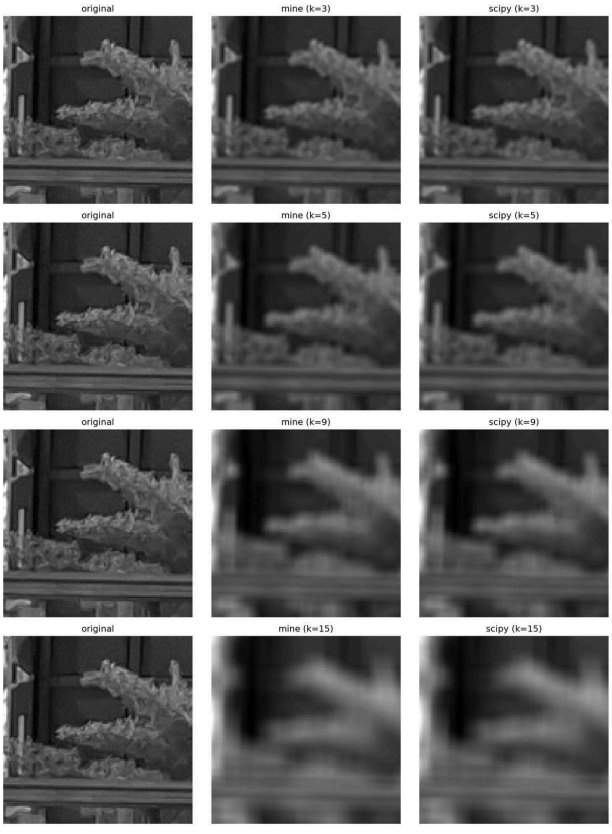
k	mine (s)	scipy (s)	max abs diff
3	0.2156	0.2152	4.44e-16
5	0.5762	0.4156	7.77e-16
9	1.8420	1.0992	2.55e-15
15	5.2441	2.7970	4.33e-15

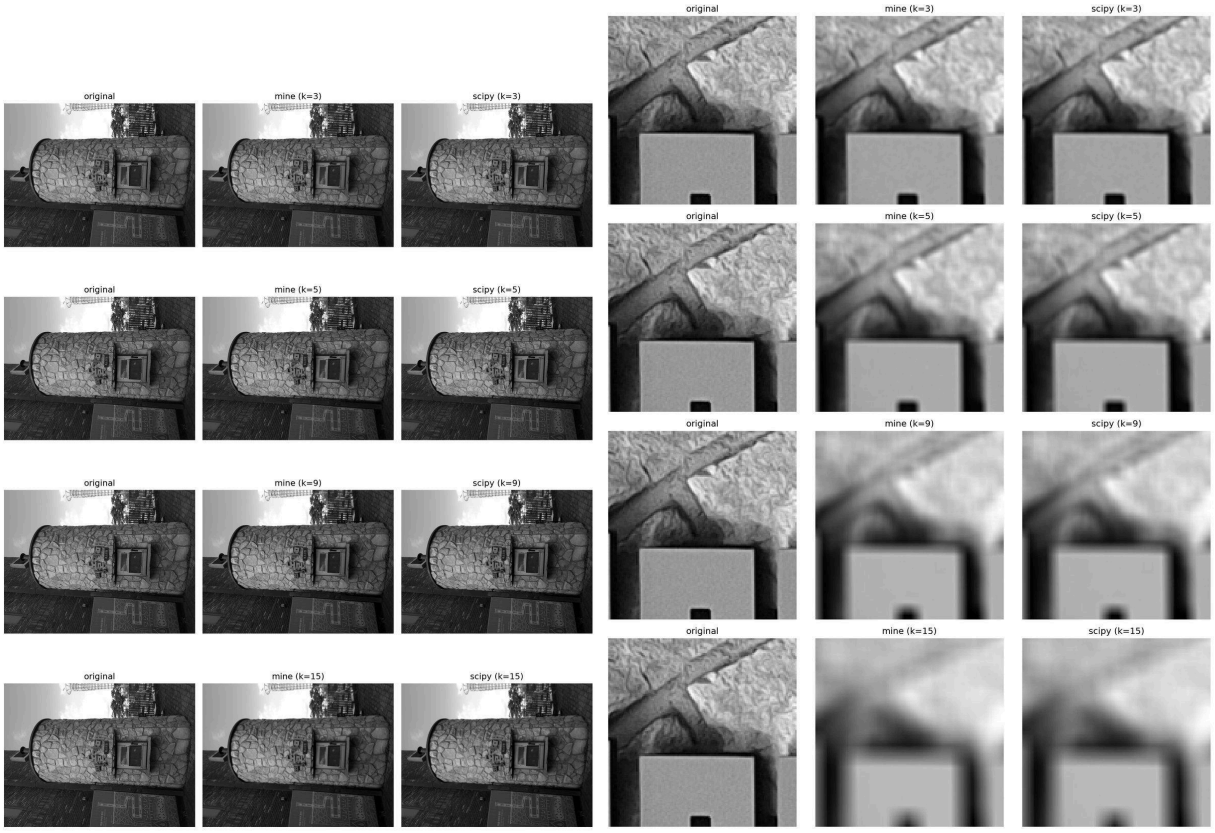
My implementation is not fast as scipy's. The difference grows as the kernel size grows.


original
mine (k=3)
scipy (k=3)
original
mine (k=5)
scipy (k=5)
original
mine (k=9)
scipy (k=9)
original
mine (k=15)
scipy (k=15)


original
mine (k=3)
scipy (k=3)
original
mine (k=5)
scipy (k=5)
original
mine (k=9)
scipy (k=9)
original
mine (k=15)
scipy (k=15)


original
mine (k=3)
scipy (k=3)
original
mine (k=5)
scipy (k=5)
original
mine (k=9)
scipy (k=9)
original
mine (k=15)
scipy (k=15)


original
mine (k=3)
scipy (k=3)
original
mine (k=5)
scipy (k=5)
original
mine (k=9)
scipy (k=9)
original
mine (k=15)
scipy (k=15)



Part 2. Gaussian Blur

Implementation. A (2r+1)×(2r+1) grid with radius r = ceil(3σ) is built, the values are computed with the Gaussian formula and normalized to sum to 1. Filtering uses the Part 1 conv2d. (script/2/part2.py)

Details.

- **Kernel size.** A Gaussian theoretically extends to infinity, but values beyond ±3σ are negligible, so it is cut at radius r = ceil(3σ). The size is 2r + 1, always odd (the assumption in conv2d). For σ = 0.5, r = 2 and the size is 5.
- **Grid.** make_grid(r) builds x and y coordinate matrices from −r to r with np.meshgrid, so the kernel formula is evaluated in one vectorized step.
- **Normalization.** After truncation, the constant $1/(2\pi\sigma^2)$ alone does not make the sum 1, so kernel / kernel.sum() is applied at the end. A sum of 1 keeps the mean brightness unchanged after blurring. This correction matters most for small σ (0.5), where a single center pixel carries most of the weight.
- **Comparison.** σ = 0.5, 1, 2, 4, 8 are applied to the same image and shown in one row. The size grows with σ, so σ = 8 is 49×49

Output. For σ = 0.5, 1, 2, 4, 8 the kernel sizes are 5, 7, 13, 25, 49. As σ grows, detail disappears and the image gets blurrier. The kernel depends only on σ and not on the image.



Part 3. Edge Detection

Implementation. Both are built on conv2d. (script/2/part3.py)

- **Sobel.** I_x and I_y come from the 3×3 kernels Sx and Sy, and the magnitude is $M = \sqrt{(I_x^2 + I_y^2)}$. The binary edge map uses the top 10% of M as the threshold (90th percentile).
- **LoG.** The kernel is a scaled LoG, $((r^2 - 2\sigma^2) / \sigma^4) \cdot \exp(-r^2 / 2\sigma^2)$, with its mean subtracted so it sums to zero. Zero-crossings of the response are found as a sign change between horizontal or vertical neighbors with a difference above 0.3. The difference threshold filters out false crossings caused by small ripples. I tried $\sigma = 1, 2, 4$.

Details.

- **Sobel kernels.** Sx = [[−1, 0, 1], [−2, 0, 2], [−1, 0, 1]] and Sy = the transpose of Sx. Each kernel smooths with [1, 2, 1] in one direction and takes a [−1, 0, 1] difference in the other. The smoothing makes it more robust to noise than a plain difference. Because conv2d flips the kernel, the signs of Ix and Iy are opposite to applying the same kernel as correlation. The magnitude M does not depend on sign, so edge detection is unaffected. Ix and Iy are displayed as gray as they are (negative values are dark).

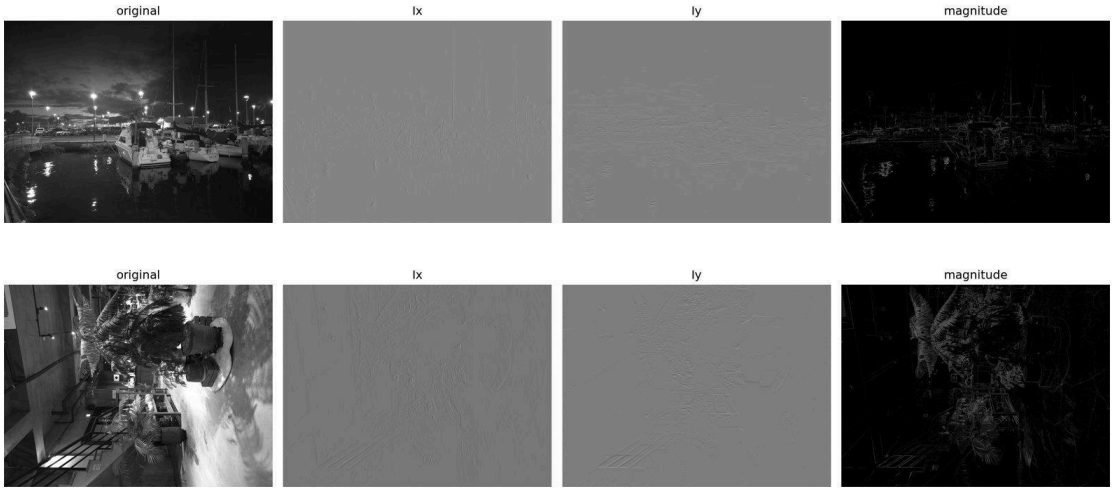
- **Sobel binarization.** M > np.percentile(M, 90) marks the top 10% of pixels as edges. A percentile is used instead of an absolute threshold so the fraction of edge pixels stays at about 10% even when brightness and contrast differ between images. The drawback is that 10% is forced even in scenes with few edges.

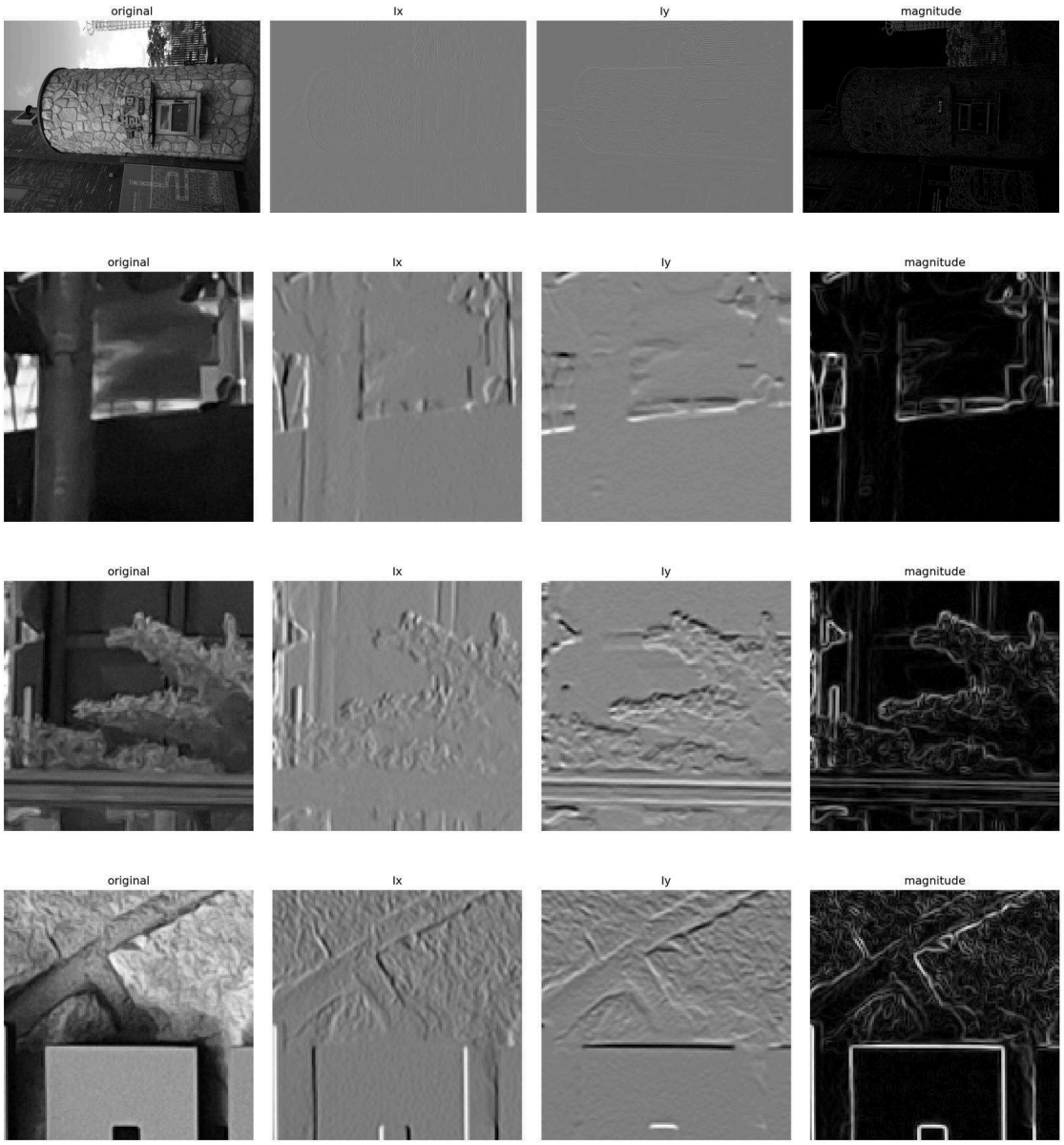
- **LoG kernel.** The formula is $((r^2 - 2\sigma^2) / \sigma^4) \cdot \exp(-r^2/2\sigma^2)$, with the same radius ceil(3σ) as the Gaussian. Truncating onto a discrete grid makes the sum non-zero, so kernel - np.mean(kernel) forces it to zero. A zero sum gives zero response in regions of constant brightness. The sign follows the formula: negative at bright points and positive where the center is dark (this is why street lamps appear as black dots in the response figure of output 2 below).

- **Zero-crossing.** For horizontal neighbors (x, x+1), a negative product of the responses means a sign change, and if the absolute difference of the two values exceeds thresh = 0.3 it counts as an edge. The vertical direction is handled the same way and the two results are OR-ed. Both pixels beside each sign change are marked (zc[:, :-1] |= ..., zc[:, 1:] |= ...), so the detected edges are about two pixels wide rather than one. The difference threshold filters out false crossings from small ripples. Because the kernel is scale-normalized ($2\pi\sigma^2 \cdot \nabla^2 G$), the peak response to an edge stays roughly the same across σ. However, a larger σ spreads the response over more pixels, so the difference between the two pixels at a crossing shrinks roughly as $1/\sigma$. A fixed threshold of 0.3 is therefore stricter at large σ, which partly explains why so few edges remain at σ = 4.

$$\nabla^2 G(x, y) \propto \frac{x^2+y^2-2\sigma^2}{\sigma^4} exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

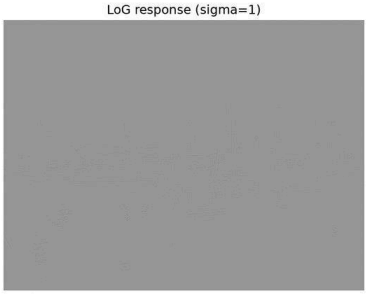
Output 1: Sobel (original, I_x, I_y, magnitude). Shown for all three images.



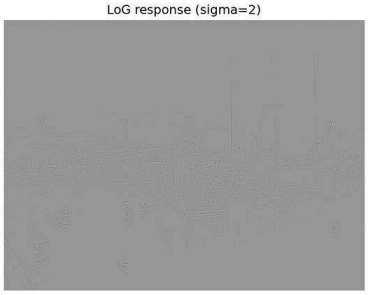


Output 2: LoG (response and zero-crossings for σ = 1, 2, 4). On example, σ = 1 picks up edges of the masts, boats and street lamps, σ = 2 cuts the edges down, and at σ = 4 only a few short edges remain.The dark sky has only weak variations, so its sign changes rarely exceed the 0.3 threshold and almost no edges are detected there. In the 128 × 128 crops, σ = 1 gives many short fragments that follow both object boundaries and noise, while σ = 4 keeps only the largest boundaries. As σ grows, edges of small details disappear and the response is dominated by large structures.

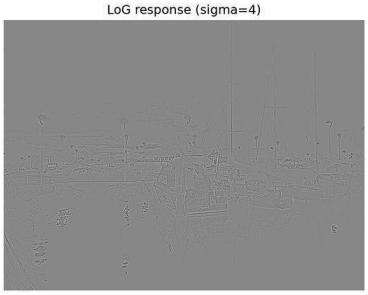
LoG response (sigma=1)



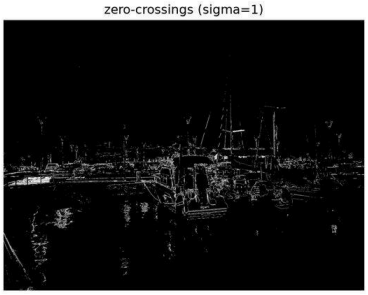
LoG response (sigma=2)



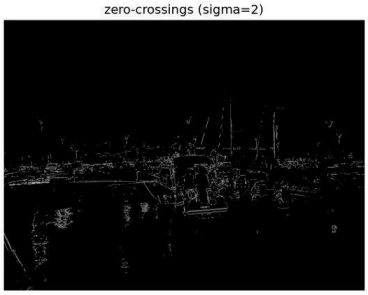
LoG response (sigma=4)



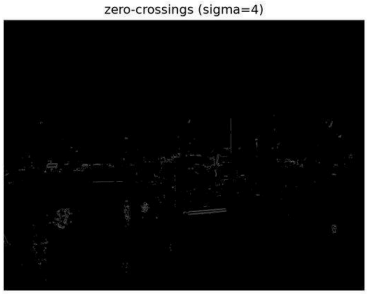
zero-crossings (sigma=1)



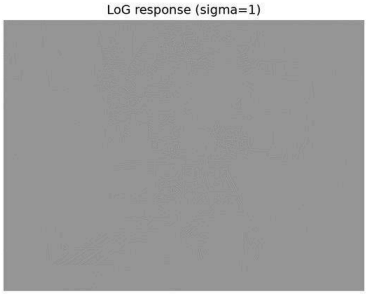
zero-crossings (sigma=2)



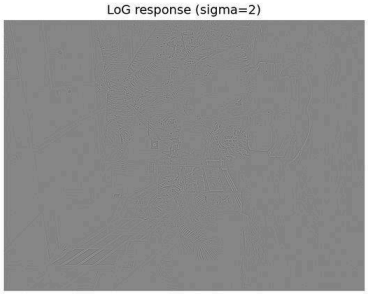
zero-crossings (sigma=4)



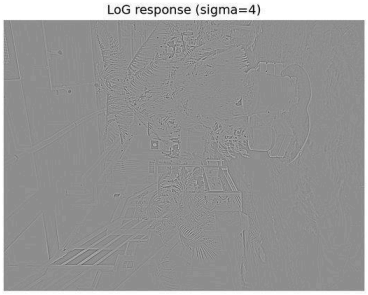
LoG response (sigma=1)



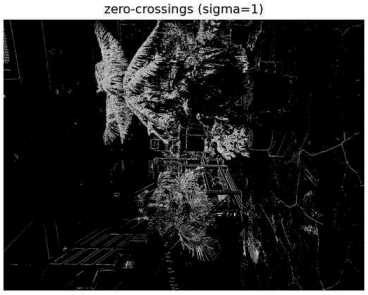
LoG response (sigma=2)



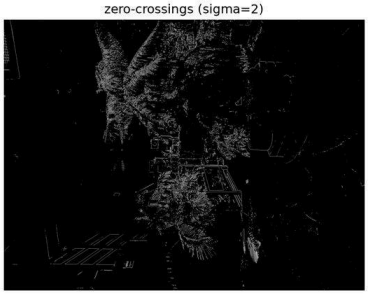
LoG response (sigma=4)



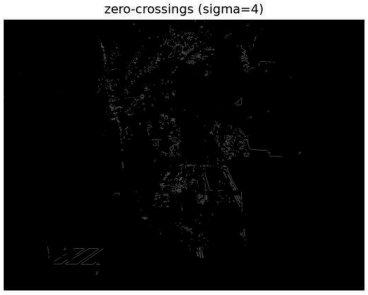
zero-crossings (sigma=1)

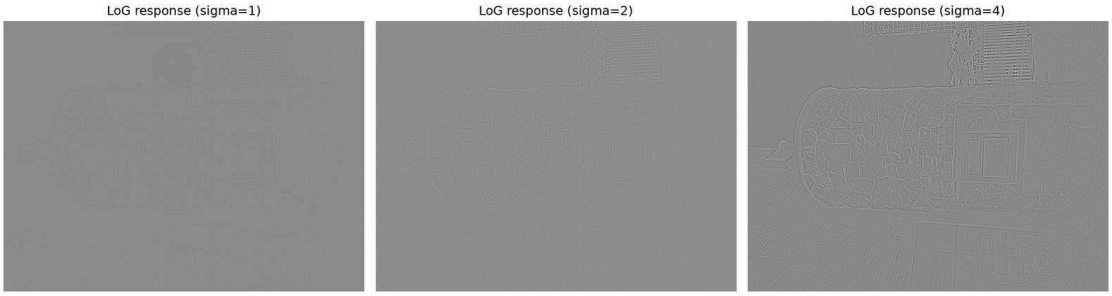


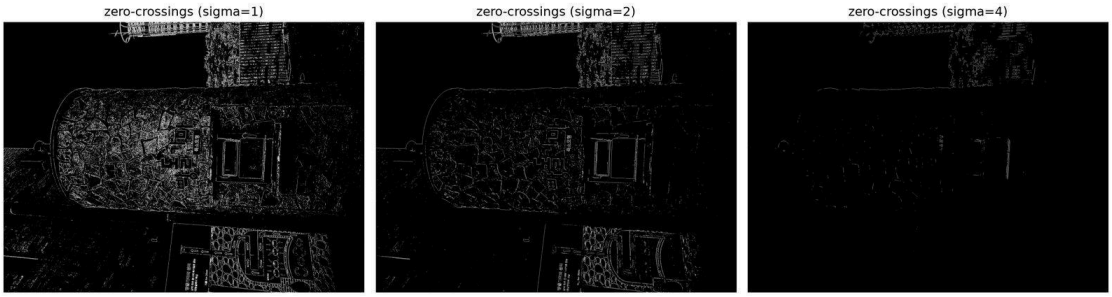
zero-crossings (sigma=2)

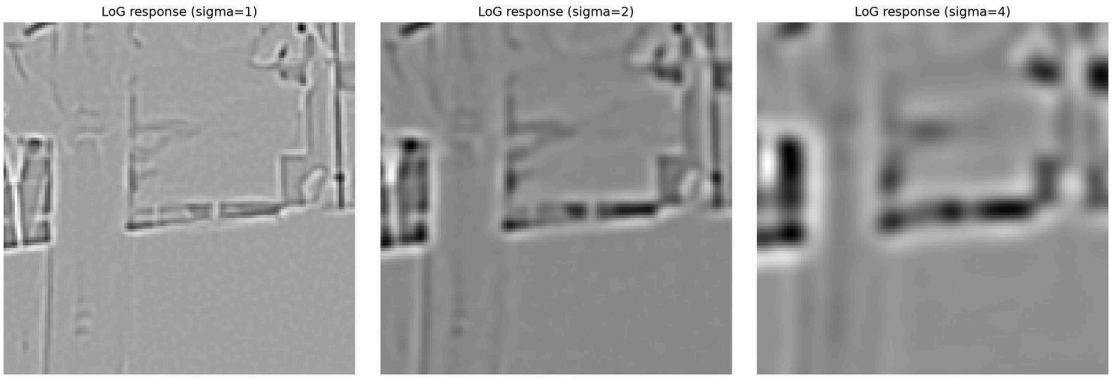


zero-crossings (sigma=4)

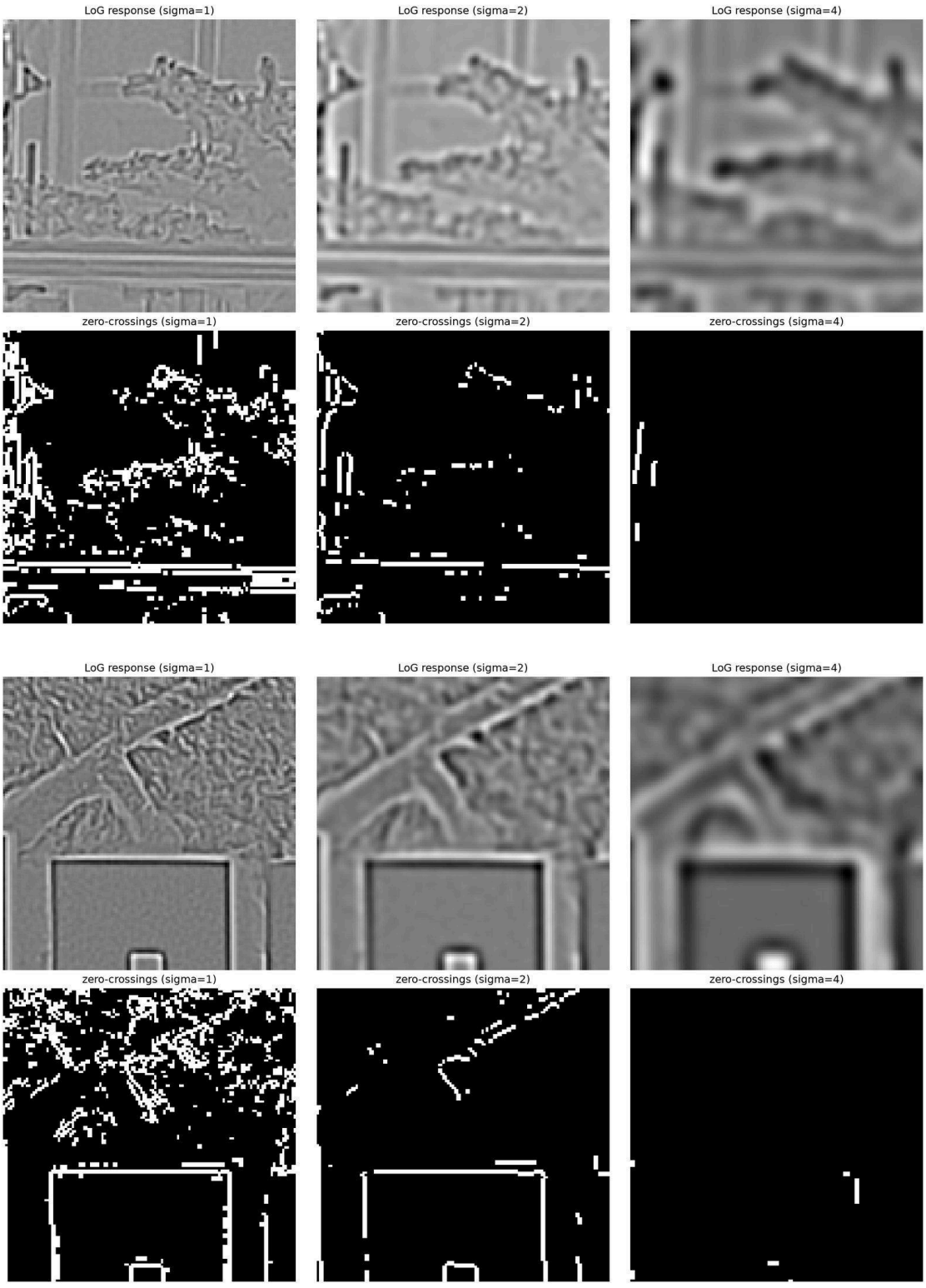



LoG response (sigma=1)
LoG response (sigma=2)
LoG response (sigma=4)


zero-crossings (sigma=1)
zero-crossings (sigma=2)
zero-crossings (sigma=4)

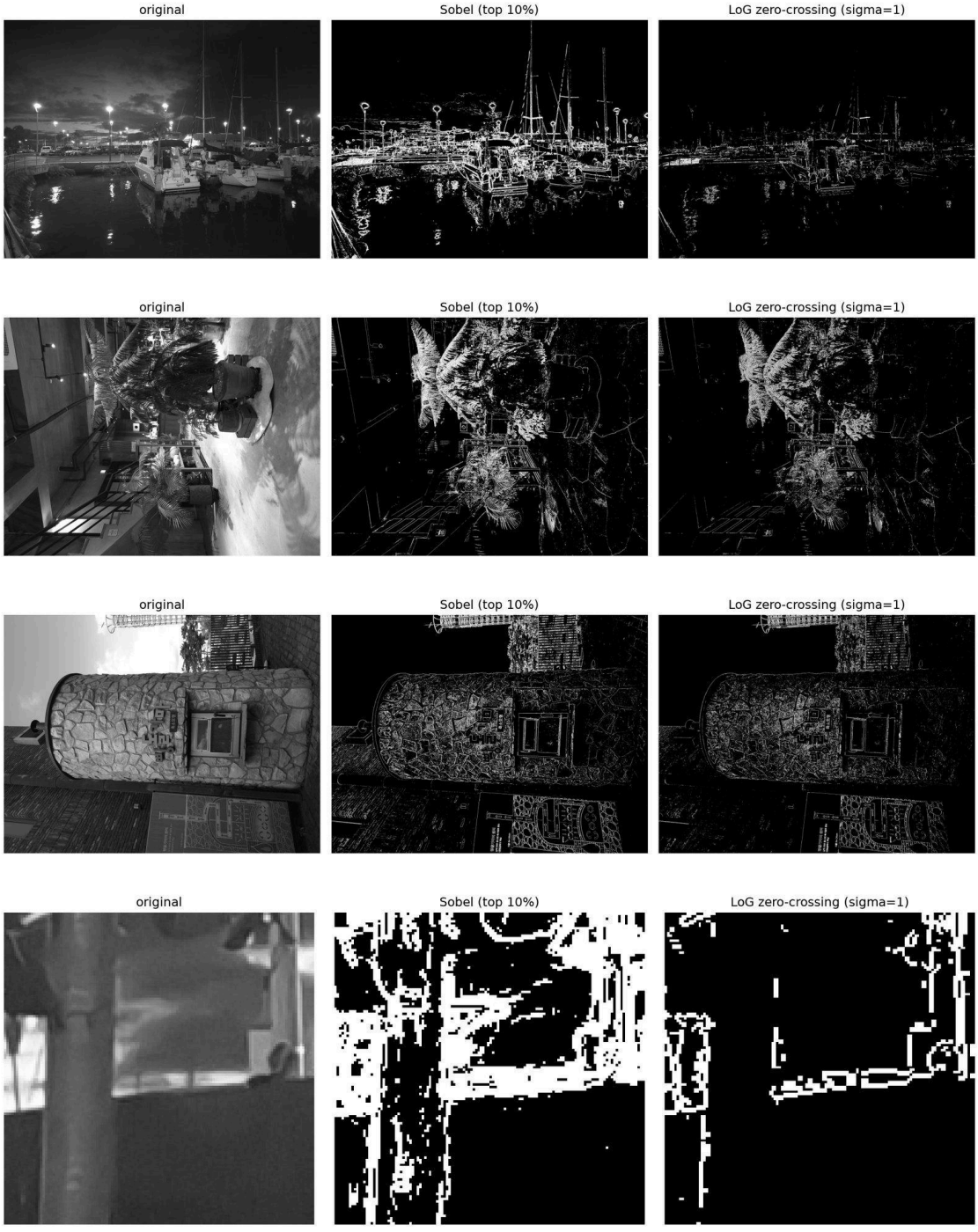

LoG response (sigma=1)
LoG response (sigma=2)
LoG response (sigma=4)

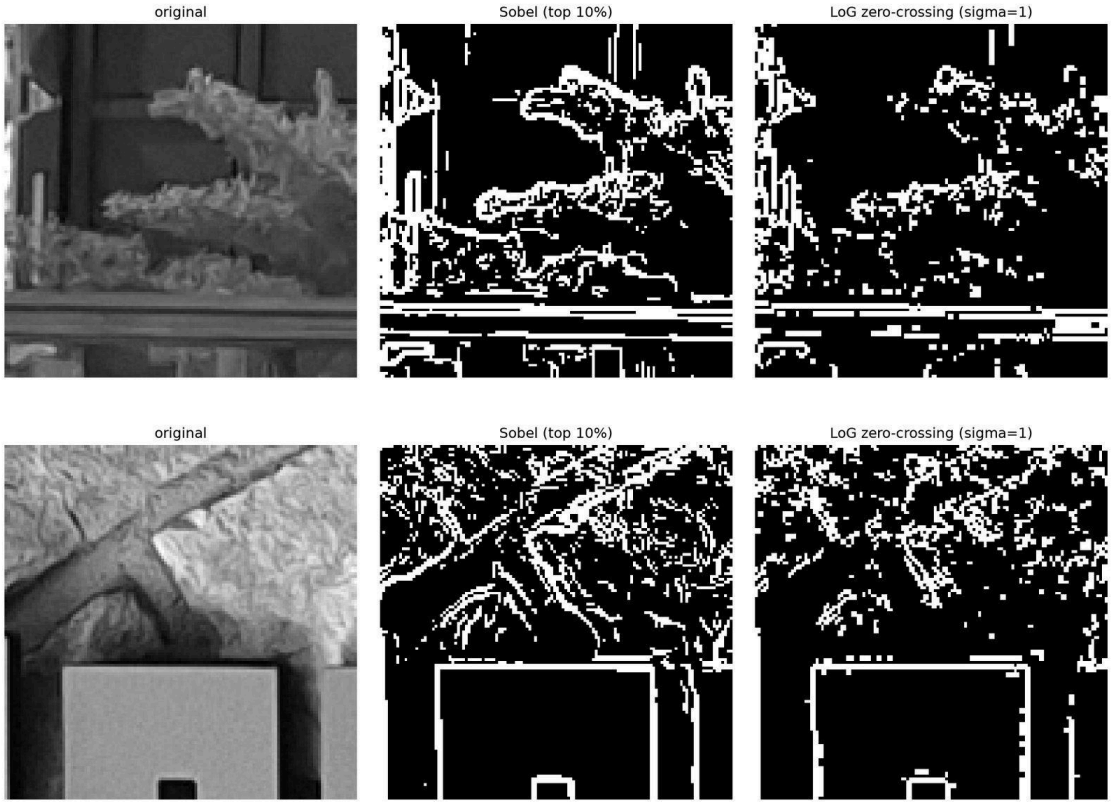

zero-crossings (sigma=1)
zero-crossings (sigma=2)
zero-crossings (sigma=4)



Output 3: Sobel vs LoG (Sobel top 10% vs LoG σ = 1 zero-crossings). Both methods capture the main structures (masts, boats, street lamps) similarly. Sobel draws thick lines where the gradient is large, and its edge fraction is fixed at about 10% by the threshold. LoG gives thinner contours (about two pixels wide, since both pixels beside each sign change are marked), but it is more sensitive to noise and its edges break into

more fragments. The central crops show the same difference at pixel level: Sobel marks wide bands around strong gradients plus scattered texture pixels, while LoG gives thin contours.





Use of AI Tools

Generative AI (Claude, GPT) was used in preparing this report: translating and formatting the text. And also used for making visualization codes.